



Technische Dokumentation

zum Projekt

MavaziHub

B2C-E-Commerce-Plattform für die Tamu Fashion GmbH

Angaben zum Dokument

Projektnummer	SWSYS-SoSe26-Gruppe08
Projektbezeichnung	MavaziHub
Auftraggeber	Tamu Fashion GmbH
Auftragnehmer	NordByte Solutions
Modul	Softwaretechnik: Systeme und Projekte (SWSYS)
Dozenten	Noah Raven und Nico Saßen
Gruppe	08
Autoren	Ayman Radi (5255058) Mahoua Sanogo (5056804) Borel Nguefack Kenfack (5386280) Baurel Junior Tanekam Kenfack (5406131)
Version	1.0
Stand	7. August 2026

Hochschule Bremen

City University of Applied Sciences

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	IV
Abkürzungsverzeichnis	V
1. Einleitung	1
1.1. Ziel dieses Dokuments	1
1.2. Projektkontext	1
1.3. Bezug zum Pflichtenheft	2
1.4. Aufbau des Dokuments	2
2. Projektplanung und Vorgehen	3
2.1. Entwicklungsprozess	3
2.2. User Stories und Mindestanforderungen	3
2.3. Aufgabenverteilung im Team	4
2.4. Verwendete Werkzeuge und Technologien	5
2.5. Versionsverwaltung und Branching-Strategie	6
2.6. Architecture Decision Records	7
3. Systemübersicht	9
3.1. Zielsystem und Gesamtsicht	9
3.1.1. Zielsystem aus Nutzersicht	9
3.2. Überblick über die Systemarchitektur	9
3.3. Zentrale Architekturentscheidungen	10
3.4. Qualitätsziele der Architektur	11
3.5. Erweiterbarkeit des Systems	13
4. Architekturbeschreibung	14
4.1. Statische Sicht	14
4.1.1. Klassendiagramm	14
4.1.2. ER-Diagramm und Datenmodell	15
5. Komponenten und Schnittstellen	17
5.1. Frontend	17
5.1.1. Frontend-Aufbau und Seitenstruktur	17
5.1.2. API-Client und Fehlerbehandlung	17
5.1.3. Authentifizierung im Frontend	18
5.1.4. Produktkatalog und Produktdetailseite	19
5.1.5. Warenkorb und Checkout	19
5.1.6. Bestellungen und Rücksendungen	20
5.1.7. Admin-Frontend	21
5.2. Backend	23
5.2.1. Datenbank und Migrationen	23
5.2.2. RESTAPI	24
5.2.3. Interne Schnittstellen	24
5.2.4. Authentifizierung und Autorisierung	25
5.2.5. Containerisierung und Systemstart	25

6. Verifikation und Tests	27
6.1. Teststrategie und Testumgebung	27
6.2. Testergebnisse	27
6.3. Verifikationsmatrix	28
7. Fazit und Ausblick	33
7.1. Zusammenfassung und Erfüllungsgrad	33
7.2. Grenzen und mögliche Erweiterungen	33
Literaturverzeichnis	35
A. Anhang	36
A.1. Architecture Decision Records	36
A.1.1. ADR-001 React/TypeScript als Frontend-Framework (SPA)	36
A.1.2. ADR-002 Spring Boot als REST-API-Backend	38
A.1.3. ADR-003 PostgreSQL als relationale Datenbank	39
A.1.4. ADR-004 Flyway für Datenbank-Migrationen	41
A.1.5. ADR-005 Vollständiger Systembetrieb über Docker Compose	42
A.1.6. ADR-006 Nginx als Webserver und Reverse Proxy im Frontend-Container . . .	44
A.1.7. ADR-007 JWT in httpOnly-Cookies statt im Browser-localStorage	45
A.1.8. ADR-008 CSRF-Schutz über Double-Submit-Cookie-Pattern	47
A.1.9. ADR-009 Persistente lokale Medienablage für Produktbilder	48
A.1.10. ADR-010 Swagger/OpenAPI für interne API-Dokumentation	49
A.1.11. ADR-011 Keine echte Zahlungs- und Versandanbindung (Simulation)	51

Abbildungsverzeichnis

4.1. ER-Diagramm des relationalen Datenmodells von MavaziHub

16

5.1. Produktübersicht und Produktdetail mit Bildergalerie

19

5.2. Gefüllter Warenkorb und Bestätigungsschritt im Checkout

20

5.3. Bestellhistorie und Bestelldetail mit Status-Timeline

21

5.4. Auswahl der Rücksendeartikel und Retourenübersicht

21

5.5. Admin-Dashboard mit Kennzahlen aus dem Demo-Datenbestand

22

5.6. Lokaler Produktbildupload und rollenbasierte Nutzerverwaltung

22

Tabellenverzeichnis

1.1. Zuordnung der Pflichtenheft-Inhalte zu den Kapiteln der technischen Dokumentation .	2
2.1. User Stories der Mindestanforderungen (Nachweis siehe Abschnitt 6.3)	4
2.2. Aufgabenverteilung im Team und Zuordnung zu den Kapiteln der technischen Dokumentation	5
2.3. Eingesetzte Werkzeuge und Technologien mit Begründung der Auswahl	6
3.1. Nutzerrollen und sichtbare Bereiche in MavaziHub	9
3.2. Architekturentscheidungen	10
3.3. Qualitätsziele, Umsetzung und Nachweise	12
3.4. Software Erweiterung	13
5.1. Wichtige Bereiche der Frontend-Struktur	17
5.2. Fachliche API-Dateien des Frontends	18
5.3. Zusammenfassung der Flyway-Migrationen	23
5.4. Interne Beziehungen zwischen Klassen	25
6.1. Ergebnisse der automatisierten Prüfungen	27
6.2. Abnahmetest des zentralen Kundenprozesses	28
6.3. Verifikationsmatrix für MavaziHub	31

Abkürzungsverzeichnis

Abkürzung	Bedeutung
API	Application Programming Interface
B2C	Business-to-Consumer
CI	Continuous Integration
CSRF	Cross-Site Request Forgery
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

1. Einleitung

1.1. Ziel dieses Dokuments

Dieses Dokument ist die technische Dokumentation (L02) für unser Projekt MavaziHub. MavaziHub ist eine E-Commerce-Plattform. Wir haben sie im Modul Softwaretechnik: Systeme und Projekte (SWSYS) an der Hochschule Bremen entwickelt.

Das Pflichtenheft (L01) zeigt, was das System können soll. Es beschreibt die Anforderungen. Diese technische Dokumentation zeigt dagegen, was wir wirklich gebaut haben. Wir erklären die Architektur, unsere wichtigsten Entscheidungen und wie wir die Anforderungen umgesetzt und geprüft haben.

Mehrere Personen lesen dieses Dokument. Für die Dozenten zeigt es, dass wir die Anforderungen aus dem Pflichtenheft richtig umgesetzt haben. Für unser eigenes Team ist es eine Referenz für die Abschlusspräsentation (L03) und die Abnahme (L04). Für spätere Entwickler – falls das Projekt weitergeht – hilft es, sich schnell in Architektur, Datenmodell und Schnittstellen einzuarbeiten, ohne den ganzen Code lesen zu müssen.

Kapitel 1 und 2 kommen von Person 1 (Mahoua Sanogo). Sie zeigen die Organisation des Projekts: Ziel, Kontext, Vorgehen, Werkzeuge und Versionsverwaltung. Die technischen Kapitel zu Architektur, Umsetzung, Datenmodell und Tests folgen in Kapitel 3 bis 7 und kommen von den anderen Teammitgliedern.

1.2. Projektkontext

Unser Auftraggeber ist die Tamu Fashion GmbH. Es ist ein Modeunternehmen aus Bremen. Die Firma verkauft traditionelle und modern interpretierte afrikanische Kleidung. Bisher verkauft die Firma nur auf informellen Wegen: persönlich in Bremen, über Social Media, E-Mail und Telefon. Das kostet viel Zeit und begrenzt die Zahl der möglichen Kunden.

Das Ziel von MavaziHub ist ein eigener Online-Shop. Kundinnen und Kunden sollen jederzeit Produkte ansehen, bestellen und den Bestellstatus verfolgen können. Mitarbeitende und Administratoren pflegen den Produktkatalog und bearbeiten Bestellungen und Rücksendungen in einem internen Verwaltungsbereich. Unser Team tritt dabei als Dienstleister NordByte Solutions auf. Wir sind für die Analyse, die Planung, die Umsetzung und die Vorbereitung der Abnahme verantwortlich.

Das System hat vier Rollen. Gäste können den Produktkatalog ansehen, aber nicht bestellen. Kunden haben nach der Registrierung zusätzlich Zugang zu Warenkorb, Bestellung, Bestellhistorie und Rücksendung. Mitarbeitende verwalten den Produktkatalog und bearbeiten Rücksendungen. Administratoren können außerdem Benutzerkonten, Rollen und Kategorien verwalten. Bestehende Kundenkonten können dabei mit Mitarbeiter- oder Administratorrechten ausgestattet werden.

Wichtig: MavaziHub ist ein Laborprojekt für die Lehrveranstaltung. Es ist kein echter, produktiver Online-Shop. Deshalb hat Version 1.0 keine echte Zahlung, keinen echten Versanddienst und kein produktives Cloud-Hosting (siehe Pflichtenheft, Abschnitt 4.7 und 5.7).

1.3. Bezug zum Pflichtenheft

Das Pflichtenheft (L01) und diese technische Dokumentation (L02) hängen eng zusammen. Das Pflichtenheft legt die Anforderungen, Anwendungsfälle, Randbedingungen und Abnahmekriterien für Version 1.0 fest. Jede Anforderung hat eine MoSCoW-Klasse (Muss, Soll, Kann, Won't) und eine Verifikationsart (Test, Inspektion, Analyse oder Review of Design).

Diese technische Dokumentation baut darauf auf. Sie zeigt, wie wir die Anforderungen umgesetzt haben, welche Entscheidungen wir dabei getroffen haben und wie wir die Umsetzung geprüft haben.

Die folgende Tabelle zeigt, wo die Themen aus dem Pflichtenheft in dieser Dokumentation wieder vorkommen:

Inhalt im Pflichtenheft (L01)	Ort in der technischen Dokumentation (L02)
Kap. 1 – Einführung, Umfeld, Zielsetzung	Kap. 1.1–1.2 (Ziel, Projektkontext)
Kap. 5.1 – Use-Case-Diagramme	Kap. 2.2 (User Stories), Kap. 4 (Architekturherleitung)
Kap. 5.2 – Funktionale Anforderungen (FA-01...FA-55)	Kapitel 5 (Komponenten und Schnittstellen) sowie Abschnitt 6.3 (Verifikationsmatrix)
Kap. 5.4/5.5 – Bedienbarkeits- & Qualitätsanforderungen	Kap. 3.4 (Qualitätsziele)
Kap. 7 – Abnahmekriterien	Kap. 6 (Verifikation und Test)
Kap. 3/4 – Voraussetzungen, Randbedingungen (Technologie-Stack)	Kap. 2.4 (Werkzeuge), Kap. 3.3 (Architekturentscheidungen)

Tabelle 1.1.: Zuordnung der Pflichtenheft-Inhalte zu den Kapiteln der technischen Dokumentation

Wichtig: Laut Laboraufgabenstellung gibt es keine Pflicht für einen direkten Vergleich zwischen L01 und dem fertigen Code. Trotzdem halten wir uns im Team an die Anforderungen aus dem Pflichtenheft. So stellen wir sicher, dass die Mindestanforderungen – Registrierung, Authentifizierung, Bestellung, Bestellhistorie und Rücksendung – vollständig umgesetzt und nachweisbar sind.

1.4. Aufbau des Dokuments

Diese technische Dokumentation besteht aus sieben Hauptkapiteln und einem ergänzenden Anhang. Kapitel 1 (Einleitung) erklärt das Projekt und den Zusammenhang mit dem Pflichtenheft. Kapitel 2 (Projektplanung und Vorgehen) zeigt, wie wir gearbeitet haben: Entwicklungsprozess, User Stories, Aufgabenverteilung, Werkzeuge, Versionsverwaltung und ein Überblick über unsere wichtigsten Entscheidungen (ADRs).

Kapitel 3 (Systemübersicht) und Kapitel 4 (Architekturbeschreibung) zeigen die technische Architektur mit Komponenten-, Klassen-, ER- und Sequenzdiagrammen. Kapitel 5 (Komponenten und Schnittstellen) beschreibt Frontend, Backend, Datenbank und Schnittstellen im Detail. Kapitel 6 (Verifikation und Tests) enthält die Teststrategie, die Testergebnisse und die Verifikationsmatrix in Abschnitt 6.3.

Kapitel 7 (Fazit und Ausblick) fasst den erreichten Projektstand zusammen und nennt Grenzen sowie mögliche Erweiterungen. Der Anhang A enthält die vollständigen Architecture Decision Records, ergänzende Diagramme, Testprotokolle und zusätzliche Screenshots.

2. Projektplanung und Vorgehen

2.1. Entwicklungsprozess

Wir haben MavaziHub Schritt für Schritt entwickelt, immer entlang der Anwendungsfälle aus dem Pflichtenheft. Zuerst haben wir gemeinsam das Pflichtenheft (L01) geschrieben. Danach haben wir die Grundstruktur aufgebaut: das Backend-Grundgerüst mit Spring Boot und das Frontend-Grundgerüst mit React. Wir haben beide Teile parallel gestartet, damit wir schnell unabhängig weiterarbeiten konnten. Danach haben wir die PostgreSQL-Datenbank eingerichtet, mit den ersten Flyway-Migrationen für Rollen und Benutzer.

Als Nächstes kam die Authentifizierung: Registrierung, Login und die Sitzungsverwaltung mit JWT. Das war wichtig, weil fast alle anderen Funktionen einen angemeldeten Kunden brauchen – zum Beispiel Warenkorb, Bestellung, Bestellhistorie und Rücksendung. Danach haben wir Schritt für Schritt den Produktkatalog gebaut (mit Varianten und Bildern), den Warenkorb, den Bestellprozess mit simulierter Zahlung und die Bestellhistorie. Parallel dazu haben wir den Rücksendungsprozess entwickelt (Rücksendung anfragen, Status ändern) und den Admin-Bereich für Produkte, Kategorien, Bestellungen und Mitarbeiterkonten.

Am Ende kam eine Stabilisierungsphase. Wir haben Randfälle, Fehlermeldungen und die Zugriffsrechte zwischen Kunde, Mitarbeiter und Administrator geprüft und verbessert. Danach haben wir das ganze System mit Docker Compose zusammengeführt. Seitdem kann man die ganze Anwendung mit einem einzigen Befehl starten: `docker compose up -build -d`. Das war uns wichtig, damit die Abnahme (L04) fair und einfach nachvollziehbar ist.

Der Ablauf zusammengefasst:

- Analyse & Pflichtenheft → Grundarchitektur (Backend- und Frontend-Gerüst, Datenbankanbindung)
- Authentifizierung & Autorisierung (Registrierung, Login, Rollenmodell)
- Produktkatalog (Kategorien, Produkte, Varianten, Bilder)
- Warenkorb & Bestellprozess (inkl. simuliertem Zahlungsstatus)
- Bestellhistorie & Rücksendungsprozess
- Admin-Verwaltungsbereich (Produkte, Kategorien, Bestellungen, Mitarbeiter/Rollen)
- Stabilisierung, Fehlerbehandlung und rollenbasierte Zugriffskontrolle
- Vollständige Containerisierung über Docker Compose und Systemtest

Wir haben uns im Team regelmäßig kurz abgesprochen. Außerdem haben wir GitHub genutzt, um Änderungen über Pull Requests nachzuvollziehen. Weil mehrere Personen an verwandten Themen gearbeitet haben (zum Beispiel Warenkorb im Frontend und Bestelllogik im Backend), haben wir die Schnittstellen – vor allem die REST-API – früh gemeinsam festgelegt. So haben wir Doppelarbeit und Fehler zwischen Frontend und Backend vermieden.

2.2. User Stories und Mindestanforderungen

Die folgende Tabelle zeigt die Mindestanforderungen aus der Laboraufgabenstellung als User Stories: Registrierung, Authentifizierung, Bestellung, Bestellhistorie und Rücksendung. Dazu kommen die passenden Verwaltungsfunktionen für Mitarbeiter und Administrator. Alle User Stories sind fertig

umgesetzt und funktionieren (Status: Erledigt). Den genauen Nachweis dafür zeigen wir in Kapitel 6, insbesondere in der Verifikationsmatrix in Abschnitt 6.3.

ID	Rolle	User Story	Priorität	Status
US-01	Gast	Als Gast möchte ich mich mit E-Mail-Adresse und Passwort registrieren, um ein Kundenkonto zu erhalten.	Muss	Erledigt
US-02	Kunde	Als Kunde möchte ich mich an- und abmelden, um geschützte Funktionen sicher nutzen zu können.	Muss	Erledigt
US-03	Gast/Kunde	Als Gast oder Kunde möchte ich den Produktkatalog durchsuchen und Produktdetails ansehen, um passende Artikel zu finden.	Muss	Erledigt
US-04	Kunde	Als Kunde möchte ich Produkte mit gewählter Variante in den Warenkorb legen und dort bearbeiten, um meinen Einkauf zusammenzustellen.	Muss	Erledigt
US-05	Kunde	Als Kunde möchte ich eine Bestellung mit Lieferadresse und Zahlungsart aufgeben, um meinen Warenkorb verbindlich zu kaufen.	Muss	Erledigt
US-06	Kunde	Als Kunde möchte ich meine Bestellhistorie einsehen, um vergangene Bestellungen nachzuvollziehen.	Muss	Erledigt
US-07	Kunde	Als Kunde möchte ich für eine Bestellung eine Rücksendung anfordern, um fehlerhafte oder ungewollte Artikel zurückzugeben.	Muss	Erledigt
US-08	Mitarbeiter	Als Mitarbeiter möchte ich Produkte, Kategorien und Varianten anlegen, bearbeiten und deaktivieren, um den Katalog aktuell zu halten.	Muss	Erledigt
US-09	Mitarbeiter	Als Mitarbeiter möchte ich eingehende Rücksendeanfragen einsehen und deren Status ändern, um Retouren zu bearbeiten.	Soll	Erledigt
US-10	Administrator	Als Administrator möchte ich bestehende Benutzerkonten aktivieren oder deaktivieren und deren Rollen verwalten, um den internen Zugriff zu steuern.	Muss	Erledigt

Tabelle 2.1.: User Stories der Mindestanforderungen (Nachweis siehe Abschnitt 6.3)

2.3. Aufgabenverteilung im Team

Wir haben die Arbeit im Team klar aufgeteilt, aber trotzdem eng zusammengearbeitet. Die Aufteilung folgt den Teilen der Anwendung (Frontend, Backend/Architektur, Daten/Tests) und den organisatorischen Aufgaben (Planung, Koordination, Zusammenbau des Dokuments). Die Schnittstellen zwischen den Bereichen – vor allem die REST-API zwischen Frontend und Backend und das Datenmodell zwischen Backend und Datenbank – haben wir gemeinsam im Team abgestimmt.

Teammitglied	Projektaufgaben	Dokumentationskapitel (L02)
Mahoua Sanogo (Person 1)	Projektplanung und -koordination, Einrichtung von Repository und Grundstruktur, Abstimmung von Werkzeugen und Git-Workflow, Zusammenbau des Gesamtdokuments.	Kap. 1, Kap. 2 sowie redaktionelle Koordination des Anhangs A
Borel Nguefack Kenfack (Person 2)	Frontend-Entwicklung mit React/TypeScript: Seitenstruktur, User Journey, Kundenfunktionen (Katalog, Warenkorb, Checkout, Bestellhistorie, Rücksendung), Admin-Oberfläche, UI-Screenshots.	Kap. 3 (anteilig), Kap. 5.1, Kap. 5.7
Baurel Junior Tanekam Kenfack (Person 3)	Systemarchitektur, Backend-Entwicklung mit Spring Boot, REST-API, Security (JWT, Rollen, CSRF), Containerisierung mit Docker Compose und Nginx.	Kap. 3, Kap. 4, Kap. 5.2, Kap. 5.4/5.5
Aymane Radi (Person 4)	Datenmodell (ER-Diagramm), Flyway-Migrationen, automatisierte Tests (JUnit, Vitest), manuelle Systemtests, Verification Matrix, Fazit und Ausblick.	Kap. 4.3, Kap. 5.3, Kap. 6, Kap. 7

Tabelle 2.2.: Aufgabenverteilung im Team und Zuordnung zu den Kapiteln der technischen Dokumentation

Alle vier Teammitglieder haben auch am Pflichtenheft (L01) mitgearbeitet. Deshalb hatten wir von Anfang an ein gemeinsames Verständnis der Anforderungen. Diese Dokumentation bewertet keine einzelnen Personen. Die Aufteilung zeigt nur, wer für welchen Bereich Auskunft geben kann.

2.4. Verwendete Werkzeuge und Technologien

Für die Entwicklung, die Projektplanung und den Betrieb von MavaziHub haben wir bekannte und gut dokumentierte Werkzeuge genutzt. Die Auswahl passt zu den technischen Vorgaben aus dem Pflichtenheft (Java Spring Boot, React mit TypeScript, PostgreSQL, Docker Compose). Wir haben auch auf gute Dokumentation und die Erfahrung im Team geachtet.

Bereich	Werkzeug	Zweck / Begründung
Frontend	React, TypeScript, Vite, CoreUI, Axios	React mit TypeScript macht die Entwicklung sicherer, weil Fehler im Code früh auffallen. Vite macht den Build schnell. CoreUI gibt fertige UI-Bausteine für die Kunden- und Admin-Oberfläche. Axios übernimmt die Kommunikation mit dem Backend.
Backend	Spring Boot 3.5, Spring Security, Spring Data JPA, Flyway, springdoc-openapi (Swagger)	Spring Boot ist im Modul vorgegeben und bietet viele fertige Bausteine für REST-APIs. Spring Security kümmert sich um Login und Rechte. Spring Data JPA macht den Datenbankzugriff einfacher. Flyway verwaltet die Änderungen am Datenbankschema. Swagger zeigt die API automatisch und hilft beim manuellen Testen.
Datenbank	PostgreSQL 16 (produktiv), H2 (Testprofil)	PostgreSQL passt gut zu unseren Daten, weil Nutzer, Produkte, Warenkorb, Bestellungen und Rücksendungen stark zusammenhängen. H2 nutzen wir nur für automatische Tests, weil es schnell und unabhängig läuft.

Bereich	Werkzeug	Zweck / Begründung
Infrastruktur	Docker, Docker Compose, Nginx	Docker Compose startet Backend, Frontend und Datenbank mit einem einzigen Befehl. So nutzen alle im Team und bei der Abnahme dieselbe Umgebung. Nginx läuft im Frontend-Container als Webserver und leitet Anfragen an /api und /media weiter.
Test	JUnit 5, Mockito, MockMvc, Vitest, Testing Library	JUnit, Mockito und MockMvc testen das Backend automatisch. Vitest und Testing Library machen das Gleiche für das React-Frontend.
Versionsverwaltung	Git, GitHub	Git und GitHub verwalten unseren Code gemeinsam. Wir sehen Änderungen über Commits und Pull Requests. GitHub ist auch der Ort, an dem wir das Projekt für L04 abgeben (siehe Kap. 2.5).
Projektmanagement	Trello (Kanban)	Trello haben wir am Anfang für Brainstorming und Aufgabenplanung genutzt, nach dem Kanban-Prinzip (Spalten To Do, In Progress, Done).
Entwicklungsumgeb	IntelliJ IDEA, VS Code, Postman	IntelliJ IDEA und VS Code haben wir für Backend bzw. Frontend genutzt. Postman haben wir zusätzlich zu Swagger benutzt, um die REST-Schnittstellen manuell zu testen.
Dokumentation (L01)	OpenAI Prism	OpenAI Prism ist eine gemeinsame Arbeitsumgebung für LaTeX. Wir haben damit das Pflichtenheft (L01) zusammen geschrieben. Alle Teammitglieder konnten gleichzeitig am selben Dokument arbeiten, mit einheitlicher Formatierung und ohne lokale LaTeX-Installation.

Tabelle 2.3.: Eingesetzte Werkzeuge und Technologien mit Begründung der Auswahl

Wir haben bewusst auf zusätzliche Werkzeuge verzichtet, zum Beispiel eine eigene CI/CD-Pipeline oder externe Monitoring-Dienste. Das liegt außerhalb des Umfangs, den das Pflichtenheft in Kapitel 4.7 festlegt. Der vollständige Start über Docker Compose (siehe Kap. 5, Docker-Abschnitt) übernimmt in unserem Projekt die Aufgabe, die in einem echten Produkt normalerweise eine CI/CD-Pipeline hätte.

2.5. Versionsverwaltung und Branching-Strategie

Für die Versionsverwaltung haben wir Git zusammen mit einem gemeinsamen GitHub-Repository genutzt.

Gemeinsame Arbeit. Wir haben nicht getrennt an eigenen Themen gearbeitet. Stattdessen haben wir bewusst im Team zusammengearbeitet, auch am selben Code. So konnte jedes Teammitglied das ganze Projekt verstehen, nicht nur den eigenen Teil.

Thematische Branches. Unsere Branches gehörten nicht einer einzelnen Person, sondern einem Thema oder einer Funktion. Zum Beispiel:

- **feature/shop-frontend-borel** – Auf diesem Branch hat das Team gemeinsam die Benutzeroberfläche (Frontend) entwickelt.
- **baurel** – Auf diesem Branch hat das Team am Backend gearbeitet.

- **shop-docker-compose** – Das war unser letzter Branch. Hier haben wir alle zusammen die Container-Integration von Backend, Frontend und Datenbank fertiggestellt.

Der Branch **main** enthielt immer den stabilen, lauffähigen Code und bildet die Grundlage für die Abgabe von L04. Wir haben nicht direkt auf **main** gepusht. Änderungen kamen zuerst auf einen Themen-Branch und wurden danach über einen Pull Request in **main** übernommen. Commit-Nachrichten haben wir möglichst klar formuliert, damit man den Verlauf später nachvollziehen kann. Einen festen Standard wie Conventional Commits haben wir aber nicht streng durchgesetzt – das beschreiben wir hier bewusst ehrlich, statt einen strengeren Prozess zu behaupten, den es in dieser Form nicht gab.

Wissensaustausch. Weil wir zusammen an denselben Themen gearbeitet haben, konnten wir uns gegenseitig helfen und Fehler früh finden. Das hat unser Projekt sehr einheitlich gemacht: Alle im Team kennen sich sowohl mit dem Frontend als auch mit dem Backend aus, nicht nur mit ihrem eigenen Bereich.

Wichtig war uns auch der Umgang mit geheimen Daten. Zugangsdaten für die Datenbank und der JWT-Schlüssel stehen nur in einer lokalen **.env**-Datei. Diese Datei laden wir nie zu GitHub hoch, weil sie in der **.gitignore**-Datei ausgeschlossen ist. Im Repository liegt nur eine **.env.example**-Datei als Vorlage, ohne echte Passwörter. Jedes Teammitglied kann daraus seine eigene **.env**-Datei erstellen (siehe README). Das passt zur Vorgabe aus dem Pflichtenheft, Abschnitt 4.3: Passwörter und geheime Schlüssel dürfen nicht offen im Repository stehen.

2.6. Architecture Decision Records

Für wichtige technische Entscheidungen nutzen wir Architecture Decision Records (ADRs). Ein ADR zeigt: Welche Möglichkeiten gab es? Welche haben wir gewählt? Warum? Und was bedeutet das für das Projekt? So können wir später nachvollziehen, warum wir eine bestimmte Lösung gewählt haben – nicht nur was wir gebaut haben.

Für MavaziHub haben wir folgende wichtige Entscheidungen als ADR festgehalten. Die vollständigen ADRs befinden sich in Abschnitt A.1 des Anhangs. Hier zeigen wir nur eine kurze Liste, damit wir uns nicht mit Kapitel 3.3 wiederholen. Dort erklären wir dieselben Entscheidungen im Zusammenhang mit der Architektur.

- ADR-01 – React als Single-Page-Application für das Frontend (statt serverseitigem Rendering)
- ADR-02 – Spring Boot als REST-API-Backend (statt eines monolithischen Server-Side-Rendering-Ansatzes)
- ADR-03 – PostgreSQL als relationale Datenbank (statt dokumentenorientierter Datenbank)
- ADR-04 – Vollständiger Betrieb des Gesamtsystems über Docker Compose (Backend, Frontend, Datenbank)
- ADR-05 – Nginx als Webserver und Reverse Proxy im Frontend-Container
- ADR-06 – Sitzungsverwaltung über JWT in httpOnly-Cookies mit CSRF-Schutz (statt Speicherung im localStorage)
- ADR-07 – Flyway zur versionierten Verwaltung des Datenbankschemas
- ADR-08 – Persistente lokale Speicherung von Produktbildern (statt Cloud-/CDN-Anbindung)
- ADR-09 – Simulierte Zahlungsabwicklung ohne Anbindung eines realen Zahlungsdienstleisters

Wir haben jede Entscheidung im Team besprochen, bevor wir sie umgesetzt haben. Die Nummern folgen der zeitlichen Reihenfolge im Projekt: zuerst die Grundentscheidungen zu Frontend und Backend,

dann die bewussten Grenzen gegenüber einem echten Produkt (ADR-08 und ADR-09). Diese Grenzen kommen direkt aus den Randbedingungen im Pflichtenheft (Kap. 4.7 und 5.7).

3. Systemübersicht

3.1. Zielsystem und Gesamtsicht

3.1.1. Zielsystem aus Nutzersicht

MavaziHub ist ein B2C-Onlineshop für die Tamu Fashion GmbH. Sichtbare Seiten und Funktionen richten sich nach Anmeldung und Rolle: Gast, Kunde, Mitarbeiter oder Administrator. So erhält jede Nutzergruppe nur die Bereiche, die sie für ihre Aufgaben benötigt.

Gäste können Startseite, Produktübersicht, Suche und Produktdetails ohne Konto nutzen. Warenkorb, Bestellungen und Rücksendungen erfordern eine Anmeldung. Danach ergänzt der Header die persönlichen Bereiche und die Abmeldung.

Tabelle 3.1.: Nutzerrollen und sichtbare Bereiche in MavaziHub

Rolle	Sichtbare Bereiche	Typische Aufgabe
Gast	Startseite, Produktliste, Suche und Produktdetails	Produkte ansehen und Informationen vergleichen
Kunde	Profil, Warenkorb, Checkout, Bestellungen und Rücksendungen	Produkt auswählen, bestellen und bei Bedarf zurücksenden
Mitarbeiter	Kundenbereiche sowie Produkt-, Bestell- und Retourenverwaltung	Produkte pflegen und laufende Bestellungen oder Retouren bearbeiten
Administrator	Kunden- und Mitarbeiterbereiche sowie Nutzer- und Rollenverwaltung	Gesamtes System verwalten und Berechtigungen vergeben

Die Rollen bauen aufeinander auf. Mitarbeiter und Administratoren behalten die Kundenfunktionen; zusätzliche Rechte weist ein Administrator einem registrierten Konto zu. Getrennte Konten sind daher nicht nötig.

Die Navigation blendet unzulässige Verwaltungsfunktionen aus, während geschützte Routen direkte Seitenaufrufe abfangen. Beides verbessert die Bedienung, ersetzt aber keine Sicherheitsprüfung: Das Backend kontrolliert jede geschützte Anfrage erneut. Die technische Umsetzung beschreibt Abschnitt 5.1.3.

3.2. Überblick über die Systemarchitektur

MavaziHub ist als containerisiertes Drei-Schichten-System umgesetzt, das vollständig über Docker Compose betrieben wird. Das System besteht aus drei Services: frontend, backend und postgres, die über ein gemeinsames Docker-Netzwerk (mavazihub-network) kommunizieren.

Die Verbindung des Nutzers erfolgt ausschließlich mit dem Frontend-Container, der über Nginx die statisch gebaute React-Anwendung ausliefert. Alle Anfragen an `/api/` und `/media/` werden von Nginx nicht selbst beantwortet, sondern per `proxy_pass` an den Backend-Service unter internem Docker-Netzwerknamen **backend:8080** weitergeleitet. Der Browser hat damit zu keinem Zeitpunkt eine direkte Verbindung zum Backend; Port 8080 wird zwar zusätzlich auf den Host gemappt, dient dort aber nur externen Werkzeugen wie Postman oder Swagger UI zu Testzwecken.

Das Spring-Boot-Backend verarbeitet die REST-Anfragen, führt die Fachlogik aus und kommuniziert seinerseits mit der PostgreSQL-Datenbank über den internen Servicenamen postgres:5432. Der Zugriff über localhost würde hier fehlschlagen, da dies innerhalb des Backend-Containers auf den Container selbst verweisen würde statt auf den Datenbank-Container. Erst die Namensauflösung durch das Docker-interne DNS über den Servicenamen ermöglicht die Verbindung.

Der Start der Services ist durch **depends_on** mit **condition: service_healthy** geregelt: Der Backend-Container startet erst, wenn der PostgreSQL-Healthcheck (**pg_isready**) erfolgreich war. Damit wird verhindert, dass Spring Boot beim Start eine Datenbankverbindung aufbauen will, während PostgreSQL noch initialisiert. Der Frontend-Container hängt wiederum vom Backend-Service ab (**depends_on: backend**), allerdings ohne Healthcheck-Bedingung, da Nginx auch dann startet, wenn das Backend noch nicht bereit ist – erste Anfragen würden in diesem Fall lediglich mit einem Fehler vom Proxy beantwortet.

3.3. Zentrale Architekturentscheidungen

Die folgenden Entscheidungen prägen die technische Umsetzung von MavaziHub grundlegend. Sie sind bewusst als Alternative-Abwägung dargestellt, ausführliche Einzelbegründungen gehören im ADR-Format. Hier werden sie kompakt zusammengefasst.

Entscheidung	Alternative	Begründung
React/Typescript als SPA	Angular	Angular wurde initial evaluiert. Als Framework für sehr große Projekte eingeschätzt und bringt mehr Struktur. React wurde stattdessen gewählt, da die Lernkurve für das Team flacher war.
Spring Boot als REST API	Node.js/Express	Team-Vorwissen im Java, ausgereiftes Ökosystem für Security, JPA und gute Integrationsfähigkeit mit PostgreSQL
PostgreSQL als relationale Datenbank	MongoDB	Fachliche Daten sind stark relational mit klaren Fremdschlüsselbeziehungen
Flyway für Schema-Migration	manuelles Schema-Management	versionierte und reproduzierbare Schema-entwicklung
Betrieb über Docker Compose	Aufsetzen jeder Komponente einzeln	keine Abhängigkeit von lokal installiert Software außer Docker
Nginx als Webserver Reverse Proxy	Direkter Zugriff des Browsers auf das Backend (CORS-basiert)	Backend bleibt aus Nutzersicht unsichtbar; SPA-routing wird zentral gelöst
JWT in httpOnly	Speicherung des Tokens im Browser	httpOnly-Cookies sind für JavaScript nicht auslesbar und robuster gegen XSS Token-diebstahl
CSRF-Schutz über Double-Submit	klassisches Spring-Security	Passt zum zustandslosen REST-API-Ansatz
Swagger/OpenAPI für interne Dokumentation	Rein manuell gepflegte API-Dokumentation	Automatisch generierte. Erleichtert Testen während der Entwicklung.

Tabelle 3.2.: Architekturentscheidungen

Die Trennung von SPA und REST-Backend zieht CORS/CSRF-Überlegungen nach sich, die vollständige Containerisierung wiederum bedingt die Nginx-Proxy-Lösung hält den Scope auf die im Pflichtenheft

geforderten Kernprozesse fokussiert.

3.4. Qualitätsziele der Architektur

Die Architektur von mavaziHub verfolgt keine einzelne, isolierte Qualitätseigenschaft, sondern balanciert mehrere Ziele, die für unser Projekt realistisch und nachweisbar sind. Jedes Ziel ist unten mit der konkreten technischen Umsetzung und einem prüfbaren Nachweis aus dem Projekt verknüpft.

Qualitätsziel	Umsetzung	Nachweis
Wartbarkeit	Klare Schichtentrennung (Controller → Service → Repository → Entity) innerhalb jedes fachlichen Moduls; einheitliches Muster über alle Module, wodurch neuer Code sich an bestehende Konventionen anlehnen kann	Modulstruktur unter <code>de.nordbyte.mavazihub</code> konsistente Namensgebung (*Controller, *Service, *Repository)
Erweiterbarkeit	Ein Modul greift auf die Funktionalität eines anderen Moduls über dessen Service zu; neue Endpunkte/Rollen lassen sich isoliert in <code>SecurityConfig</code> ergänzen	Rückgabefähigkeits-Logik in <code>OrderHistoryService</code> nutzt <code>ReturnItemRepository</code> , ohne das <code>returnrequest</code> -Modul selbst zu verändern
Sicherheit	Rollenbasierte Zugriffskontrolle zentral in <code>SecurityConfig</code> ; JWT in <code>httpOnly-Cookies</code> gegen XSS; Double-Submit-CSRF-Schutz gegen Cross-Site-Request-Forgery; serverseitige Neuprüfung von Kontostatus (gesperrt/deaktiviert) bei jedem Request über <code>JwtAuthenticationFilter</code>	<code>SecurityConfig.authorizeHttpRequests</code> , <code>CsrfValidationFilter</code> , <code>JwtAuthenticationFilter.isActive()</code>
Testbarkeit	Fachlogik ist in Services statt in Controllern konzentriert; DTOs entkoppeln API-Verträge von internen Entities; dadurch lassen sich Services isoliert per Unit-Test und Endpunkte per <code>MockMvc</code> -Integrationstest prüfen, ohne den vollen Stack starten zu müssen	Backend-Testklassen
Reproduzierbarkeit	Vollständiger Systemstart über eine einzige <code>docker-compose.yml</code> mit drei Services; Datenbankschema wird nicht implizit von Hibernate verändert (<code>ddl-auto: validate</code>), sondern ausschließlich versioniert über Flyway-Migrationen angewendet	Verifizierter Ablauf <code>docker compose down → docker compose up -build -d</code> ; Flyway-Migrationsverzeichnis <code>db/migration</code>
Verfügbarkeit beim Start	PostgreSQL-Healthcheck verhindert, dass das Backend startet, bevor die Datenbank tatsächlich Verbindungen annimmt; <code>depends_on: condition: service_healthy</code> erzwingt die korrekte Startreihenfolge	<code>docker-compose.yml</code> , Abschnitt <code>postgres.healthcheck</code> und <code>backend.depends_on</code>
Nachvollziehbarkeit	Automatisch generierte API-Dokumentation über Swagger/OpenAPI direkt aus dem Code; strukturierte, einheitliche Fehlerantworten über <code>GlobalExceptionHandler</code>	Erreichbarkeit von <code>/swagger-ui/**</code> und <code>/v3/api-docs/**</code> ; <code>ApiError-Record</code> mit Zeitstempel, Status, Fehlertext und aufgerufenem Pfad

Es muss erwähnt werden, dass einige Qualitätsziele bewusst nicht verfolgt sind.

- Keine horizontale Skalierbarkeit im Sinne einer Microservice-Architektur: Das Backend ist ein monolithischer Spring-Boot-Dienst, da der Projektumfang keine verteilte Architektur rechtfertigt.
- Keine Produktionsreife: Cookies laufen im lokalen HTTP-Betrieb ohne *Secure*-Flag, es gibt keine Lasttests, keine rechtliche Vollprüfung und keine Penetrationstests.
- Keine automatische Skalierung der Medienablage: Produktbilder liegen in einem lokalen Docker-Volume statt in einem CDN.

3.5. Erweiterbarkeit des Systems

Der Fokus liegt selbst bewusst auf der Erweiterung der Architektur ohne den bestehenden Module grundlegend umzubauen.

Erweiterung	betroffene Schicht	Begründung
Wunschliste, Produktbewertung	Neues Modul nach bestehenden Mustern	bestehende Vorlage dienen als Vorlage, keine Änderung an bereits vorhandenen Modulen nötig.
Austausch der Medienstrategie	ProductImageStorageService, MediaWebConfig, MediaStorageProperties	Speicherlogik ist bereits in einem eigenen Service gekapselt
Integration eines echten Zahlungsanbieters	Neues Modul/Service, Erweiterung Order-entity um echten Zahlungsstatus	Aktuell wird der Zahlungsstatus lokal simuliert; eine echte Anbindung erfordert asynchrone Callback-Verarbeitung (Webhooks) und zusätzliche Fehlerbehandlung, die im aktuellen synchronen Checkout-Fluss noch nicht vorgesehen ist

Tabelle 3.4.: Software Erweiterung

4. Architekturbeschreibung

4.1. Statische Sicht

4.1.1. Klassendiagramm

Das Klassendiagramm zeigt die zentralen Domänenklassen des MavaziHub-Backends. Die Darstellung basiert auf den implementierten JPA-Entities und konzentriert sich auf die fachlichen Bereiche Benutzerverwaltung, Produktkatalog, Warenkorb, Bestellungen und Rücksendungen. Technische Klassen wie Controller, Services, Repositories, DTOs und Sicherheitsfilter werden zugunsten der Übersicht nicht dargestellt.

Die Benutzerverwaltung wird hauptsächlich durch die Klassen `User`, `Role` und `RefreshToken` abgebildet. Ein Benutzer kann mehrere Rollen besitzen. Dadurch kann ein Konto beispielsweise gleichzeitig normale Kundenfunktionen und zusätzliche Mitarbeiter- oder Administratorrechte erhalten. Ein Refresh-Token ist dagegen genau einem Benutzer zugeordnet und unterstützt die Erneuerung einer angemeldeten Sitzung.

Der Produktkatalog besteht aus `Category`, `Product`, `ProductVariant` und `ProductImage`. Jedes Produkt gehört zu einer Kategorie und kann mehrere Varianten sowie mehrere Bilder besitzen. Varianten bilden konkrete Ausprägungen eines Produktes, beispielsweise durch Größe, Farbe oder Muster, und besitzen einen eigenen Lagerbestand.

Eine Warenkorbposition wird durch `CartItem` repräsentiert. Sie enthält die Kennung des Kunden, des Produktes und optional einer Variante sowie Menge und Einzelpreis. Die Verbindungen zu Benutzer und Produkt werden dabei über Identifikatoren und nicht durch direkte JPA-Beziehungen hergestellt. Dies verringert die Kopplung zwischen den Fachmodulen. Die Gültigkeit der Identifikatoren wird durch die Geschäftslogik geprüft.

Die Bestellverwaltung verwendet die Klassen `Order` und `OrderItem`. Zwischen ihnen besteht eine Eins-zu-Viele-Beziehung, da eine Bestellung mehrere Bestellpositionen enthalten kann. Produktname, Einzelpreis und Lieferadresse werden zum Zeitpunkt der Bestellung als Momentaufnahme gespeichert. Spätere Änderungen an einem Produkt oder am Benutzerprofil verändern dadurch keine bereits aufgegebene Bestellung.

Rücksendungen werden durch `ReturnRequest` und `ReturnItem` modelliert. Eine Rücksendeanfrage gehört zu einer Bestellung und kann mehrere zurückzusendende Positionen enthalten. Jede Rücksendeposition verweist auf eine Position der ursprünglichen Bestellung und enthält die gewünschte Rücksendemenge.

Die Entity-Klassen werden nicht direkt über die REST-Schnittstelle übertragen. Controller verwenden Request- und Response-DTOs, während die fachliche Verarbeitung in Services erfolgt. Repositories übernehmen anschließend den Datenbankzugriff. Der interne Ablauf folgt damit vereinfacht der Struktur:

`Controller` → `DTO` → `Service` → `Repository` → `Entity`

Durch diese Schichtentrennung bleiben REST-Schnittstelle, Geschäftslogik und Persistenz voneinander abgegrenzt und können unabhängig getestet oder weiterentwickelt werden. Die Übertragung der Klassen in das relationale Datenbankschema wird im folgenden Abschnitt anhand des ER-Diagramms erläutert.

4.1.2. ER-Diagramm und Datenmodell

Das ER-Diagramm in Abbildung 4.1 zeigt die relationale Struktur der persistenten Daten von MavaziHub. Das aktuelle Schema umfasst 13 fachliche Tabellen aus den Bereichen Benutzerverwaltung, Authentifizierung, Produktkatalog, Warenkorb, Bestellung und Rücksendung. Technische Tabellen, die beispielsweise von Flyway selbst angelegt werden, sind nicht Bestandteil der Darstellung.

Für die Identifikation der Datensätze werden zwei Schlüsselarten verwendet. Benutzer-, Authentifizierungs- und Transaktionsdaten wie `users`, `refresh_tokens`, `cart_item`, `orders`, `order_items`, `return_request` und `return_item` verwenden UUIDs. Die Tabellen des Produktkatalogs und die Rollenverwaltung verwenden dagegen automatisch erzeugte numerische Schlüssel vom Typ `BIGINT`.

Die Benutzer- und Rollenverwaltung wird durch die Tabellen `users`, `roles` und `users_roles` abgebildet. Zwischen Benutzern und Rollen besteht eine Viele-zu-Viele-Beziehung, die durch die Zuordnungstabelle `users_roles` umgesetzt wird. Ein Benutzer kann dadurch mehrere Rollen besitzen, während dieselbe Rolle mehreren Benutzern zugewiesen werden kann. Die Tabelle `refresh_tokens` steht in einer Viele-zu-Eins-Beziehung zu `users`, da ein Benutzer im zeitlichen Verlauf mehrere Refresh-Tokens besitzen kann.

Der Produktkatalog besteht aus `categories`, `products`, `product_variants` und `product_images`. Eine Kategorie kann mehrere Produkte enthalten, während jedes Produkt genau einer Kategorie zugeordnet ist. Ein Produkt kann außerdem mehrere Varianten und mehrere Bilder besitzen. Varianten speichern konkrete Ausprägungen wie Größe, Farbe oder Muster zusammen mit ihrem eigenen Lagerbestand.

Die Bestellverwaltung verwendet die Tabellen `orders` und `order_items`. Eine Bestellung enthält eine oder mehrere Bestellpositionen. Die Lieferadresse, der Produktname und der Einzelpreis werden beim Erstellen der Bestellung als Momentaufnahme gespeichert. Änderungen am Benutzerprofil oder am Produktkatalog wirken sich daher nicht nachträglich auf bereits aufgegebene Bestellungen aus.

Rücksendungen werden durch `return_request` und `return_item` abgebildet. Eine Rücksendeanfrage gehört zu einer Bestellung und kann mehrere Rücksendepositionen enthalten. Jede Rücksendeposition verweist auf eine Position der ursprünglichen Bestellung und enthält die zurückzusendende Menge.

Das Datenmodell unterscheidet zwischen physischen Fremdschlüsseln und fachlichen Referenzen. Eng zusammengehörende Datensätze, beispielsweise Bestellung und Bestellposition oder Produkt und Produktbild, werden durch Fremdschlüssel abgesichert. Modulübergreifende Bezüge wie `customer_id`, `product_id` oder `variant_id` werden teilweise nur als Identifikatoren gespeichert. Ihre Gültigkeit wird durch die Services des Backends geprüft. Dadurch bleiben die Fachmodule voneinander entkoppelt, ohne dass der fachliche Zusammenhang verloren geht.

Das ER-Diagramm beschreibt den resultierenden relationalen Aufbau. Wie dieses Schema mit PostgreSQL und den versionierten Flyway-Migrationen erzeugt wird, wird in Abschnitt 5.2.1 erläutert.

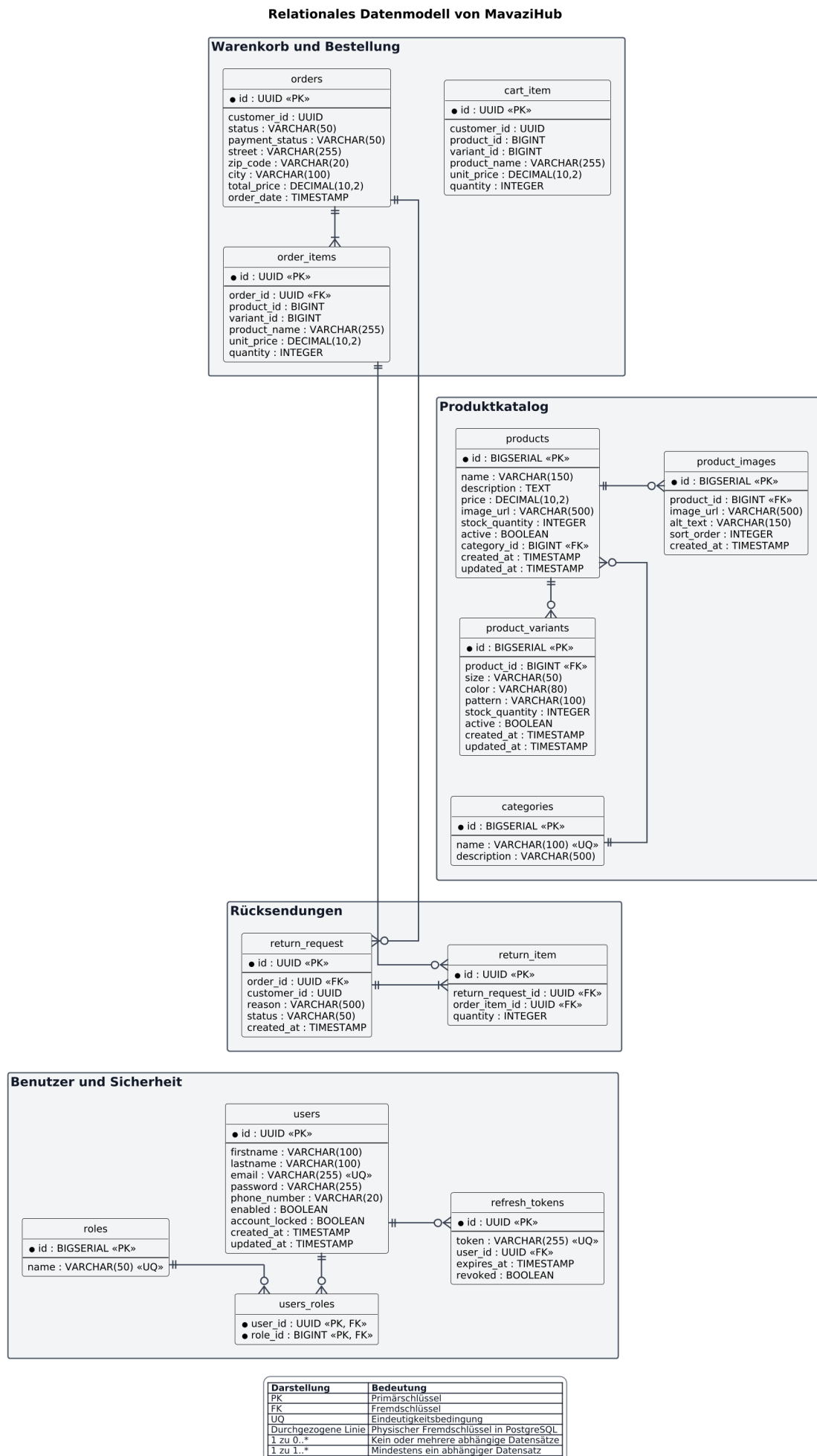


Abbildung 4.1.: ER-Diagramm des relationalen Datenmodells von MavaziHub

5. Komponenten und Schnittstellen

5.1. Frontend

Das Frontend bildet die sichtbare Oberfläche von MavaziHub. Es stellt sowohl den öffentlichen Shop als auch die geschützten Kunden- und Verwaltungsbereiche bereit. Die Oberfläche wurde als Single-Page-Application mit React und TypeScript umgesetzt. React teilt die Oberfläche in einzelne Komponenten auf und aktualisiert nur die Teile, deren Zustand sich geändert hat. TypeScript ergänzt JavaScript um feste Typen. Dadurch werden zum Beispiel unvollständige Produkt- oder Bestelldaten bereits während der Entwicklung erkannt.

5.1.1. Frontend-Aufbau und Seitenstruktur

Der Frontend-Quellcode unter `frontend/frontend/src` ist nach Aufgaben gegliedert. Vollständige Ansichten liegen in `pages`, wiederverwendbare Elemente in `components`. Dadurch kann etwa die Status-Timeline in mehreren Bestell- und Retourenansichten eingesetzt werden.

Tabelle 5.1.: Wichtige Bereiche der Frontend-Struktur

Ordner oder Datei	Aufgabe	Beispiele
<code>pages</code>	Vollständige Seiten, die über eine URL aufgerufen werden	Produktübersicht, Warenkorb, Checkout, Bestellungen und Adminseiten
<code>components</code>	Wiederverwendbare Teile der Oberfläche und gemeinsame Layouts	Header, Footer, Produktkarte, Produktbild und Status-Timeline
<code>api</code>	Kommunikation mit den REST-Schnittstellen des Backends	Produkt-, Warenkorb-, Bestell-, Retouren- und Admin-API
<code>auth</code>	Verwaltung des angemeldeten Nutzers und Schutz von Seiten	<code>AuthContext</code> und <code>ProtectedRoute</code>
<code>types</code>	Gemeinsame TypeScript-Typen für Anfragen und Antworten	<code>ProductResponse</code> , <code>CartResponse</code> und <code>OrderResponse</code>
<code>App.tsx</code>	Zuordnung der URLs zu Seiten und Rollen	<code>/products</code> , <code>/orders</code> und <code>/admin</code>

`App.tsx` ordnet URLs den Seiten zu. Startseite, Produktkatalog, Produktdetails, Login und Registrierung sind öffentlich. Warenkorb, Checkout, Profil, Bestellungen und Rücksendungen benötigen eine Anmeldung. Der Verwaltungsbereich erlaubt `ROLE_EMPLOYEE` und `ROLE_ADMIN`; die Nutzerverwaltung ist Administratoren vorbehalten.

`ShopLayout` verbindet Header, Inhalt und Footer. `AdminLayout` bietet dagegen eine Seitennavigation für Tabellen und Formulare. Beide Bereiche bleiben Teil desselben React-Projekts, sind aber an die unterschiedlichen Aufgaben von Kunden und Beschäftigten angepasst.

5.1.2. API-Client und Fehlerbehandlung

Das Frontend kommuniziert über eine gemeinsame Axios-Instanz in `axiosClient.ts` mit dem Backend. Die Basisadresse `/api` vermeidet wiederholte Serveradressen und Header. In Docker Compose leitet Nginx diese Anfragen an Spring Boot weiter; andere Umgebungen können `VITE_API_BASE_URL` setzen.

Mit `withCredentials` sendet der Browser die Anmelde-Cookies automatisch. Bei schreibenden Anfragen ergänzt ein Request-Interceptor den Header `X-CSRF-Token`. Nach einer 401-Antwort versucht

Tabelle 5.2.: Fachliche API-Dateien des Frontends

API-Datei	Fachlicher Zweck	Beispiel einer Anfrage
<code>authApi.ts</code>	Registrierung, Anmeldung, Aktualisierung und Abmeldung	POST <code>/auth/login</code>
<code>productApi.ts</code>	Öffentlicher Produktkatalog, Details und Varianten	GET <code>/products/{id}</code>
<code>cartApi.ts</code>	Warenkorbpositionen und Abschluss einer Bestellung	POST <code>/cart/items</code>
<code>orderApi.ts</code>	Eigene Bestellungen und bestellbare Positionen	GET <code>/orders/my</code>
<code>returnApi.ts</code>	Rücksendung anfordern und eigene Rücksendungen laden	POST <code>/returns</code>
<code>adminApi.ts</code>	Produkte, Bilder, Varianten, Nutzer, Bestellungen und Retouren verwalten	PATCH <code>/admin/orders/{id}/status</code>
<code>categoryApi.ts</code>	Öffentliche Kategorien für die Produktfilterung	GET <code>/categories</code>
<code>userApi.ts</code>	Eigenes Profil und Passwort bearbeiten	PUT <code>/users/me</code>

der Response-Interceptor einmal, die Sitzung über `/auth/refresh` zu erneuern und wiederholt anschließend die Anfrage. Eine gemeinsame laufende Refresh-Anfrage verhindert parallele Erneuerungen; schlägt sie fehl, wird der Nutzer abgemeldet. Die serverseitige Token- und CSRF-Prüfung wird im Backend-Abschnitt zur Authentifizierung beschrieben.

Die Seiten übersetzen technische Antworten in verständliche Meldungen. Sie unterscheiden unter anderem eine abgelaufene Sitzung, ungültige Auswahl, fehlenden Bestand, leeren Warenkorb und Serverprobleme. Interne Endpunktnamen oder Stacktraces werden nicht angezeigt; fehlt eine genaue Ursache, erscheint eine neutrale allgemeine Meldung.

5.1.3. Authentifizierung im Frontend

Der `AuthContext` verwaltet den Anmeldestatus zentral. Beim Start ruft er `/users/me` auf und speichert bei gültiger Sitzung Nutzer und Rollen im React-Zustand. Schlägt die Anfrage fehl, gilt die Anwendung als nicht angemeldet.

Access- und Refresh-Token liegen als `HttpOnly`-Cookies vor und sind für Frontend-JavaScript nicht lesbar. Der Browser sendet sie automatisch. Nur das CSRF-Cookie wird gelesen und zusätzlich im Request-Header übertragen. Für die lokale HTTP-Umgebung gelten `SameSite=Lax` und kein `Secure`-Attribut; bei HTTPS muss `Secure` aktiviert werden. Die Erzeugung und Prüfung der Tokens erfolgt im Backend.

`ProtectedRoute` setzt den in Abbildung ?? dargestellten Ablauf um. Während der Nutzer geladen wird, erscheint ein Ladezustand. Ohne Anmeldung folgt die Weiterleitung zum Login; bei geschützten Bereichen wird zusätzlich mindestens eine erforderliche Rolle geprüft. Derselbe Zustand steuert den Header: Bestellungen und Rücksendungen erscheinen nach der Anmeldung, der Arbeitsbereich nur für Mitarbeiter und Administratoren. Beim Logout löscht das Backend die Cookies; anschließend werden Nutzerzustand und Navigation aktualisiert.

Diese Prüfungen unterstützen Navigation und Bedienung, sind aber keine Sicherheitsgrenze. Das Backend kontrolliert Anmeldung, Kontostatus und Rolle bei jeder geschützten Anfrage erneut.

5.1.4. Produktkatalog und Produktdetailseite

Die Produktübersicht lädt Kategorien und Produkte parallel. Kategorien und ein Suchfeld filtern nach Produktname, Kategorie, Material und Beschreibung. Suchbegriff und Kategorie stehen als URL-Parameter zur Verfügung, sodass Filter nach dem Neuladen erhalten und teilbar bleiben. Beim Laden erscheinen Skeleton-Karten.

Produktkarten zeigen Bild, Marketing-Hinweis, Kategorie, Name, Material und Preis. Hinweise wie "Beliebt", "Bestseller", "Limitierte Edition" oder "Neu" werden in der Laborversion regelbasiert aus der Produkt-ID gewählt, statt erfundene Verkaufszahlen zu verwenden. Später können gepflegte Merkmale oder echte Auswertungen diese Regel ersetzen.

ProductVisual zeigt bei fehlenden oder nicht erreichbaren Bildern einen Platzhalter mit Produktname und Kategorie. Ist der Produktserver vorübergehend nicht erreichbar, kann die Übersicht lokale Demodaten anzeigen. Dieser Präsentations-Fallback gilt nur für den Katalog; Warenkorb, Bestellung und Rücksendung benötigen weiterhin Backend-Daten.

Die Detailseite führt alle Bildadressen zu einer Galerie zusammen. Das erste Bild ist das Hauptbild, weitere lassen sich über Vorschaubilder wählen. Varianten wie Größe, Farbe oder Muster erscheinen als Auswahl; deaktivierte oder ausverkaufte Varianten sind nicht wählbar. Kunden sehen statt interner Bestandszahlen die Zustände "Auf Lager", "Nur noch wenige verfügbar" und "Ausverkauft".

Vor dem Hinzufügen prüft die Seite Variante und Menge. Diese kann den gemeldeten Bestand nicht überschreiten. Nach Erfolg erscheint ein Link zum Warenkorb; Fehler unterscheiden fehlende Anmeldung, ungültige Auswahl und nicht verfügbare Menge.

Abbildung 5.1 zeigt Produktübersicht und Detailseite mit Suche, Varianten, Galerie und Warenkorbsteuerung.

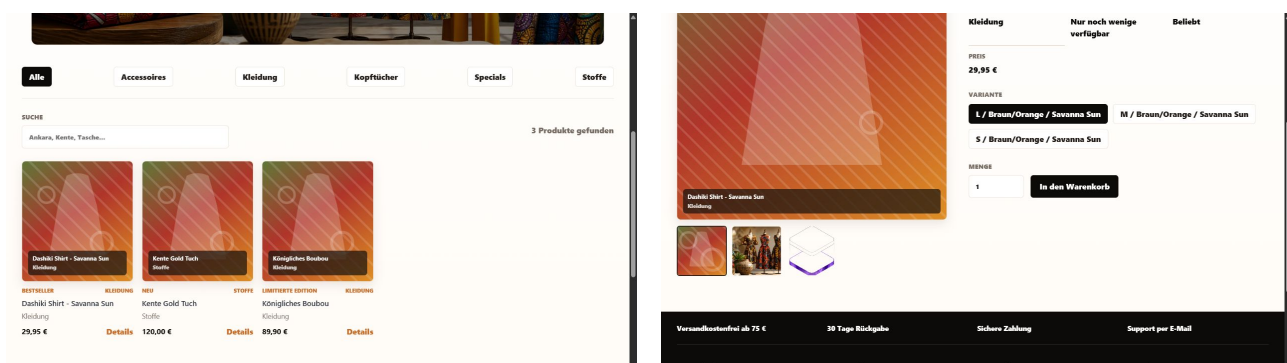


Abbildung 5.1.: Produktübersicht und Produktdetail mit Bildergalerie

5.1.5. Warenkorb und Checkout

Der Warenkorb ist nur für angemeldete Nutzer erreichbar. Beim Öffnen lädt die Seite den aktuellen Warenkorb aus dem Backend. Für jede Position werden Produktname, Einzelpreis, Menge, Zwischensumme und gegebenenfalls die Varianten-ID angezeigt. Die Menge kann mit Plus- und Minus-Schaltflächen oder direkt im Eingabefeld geändert werden. Eingaben kleiner als eins werden abgelehnt. Artikel können einzeln entfernt oder der gesamte Warenkorb kann geleert werden.

Nach jeder Änderung verwendet das Frontend die vollständige Warenkorb-Antwort des Backends. Preis und Summe werden daher nicht nur im Browser weitergerechnet, sondern mit dem serverseitigen Stand abgeglichen. Während eine Position gespeichert wird, sind ihre Bedienelemente vorübergehend deaktiviert. Damit werden doppelte Anfragen durch schnelles mehrfaches Klicken reduziert. Bei einem leeren Warenkorb erscheint eine eigene Ansicht mit einem Link zurück zur Produktübersicht.

Der Checkout besteht aus vier sichtbaren Schritten: Warenkorb, Lieferadresse, Zahlungsart und Bestätigung. Zuerst werden die ausgewählten Artikel nochmals zusammengefasst. Danach gibt der Nutzer Straße, Postleitzahl und Ort ein. Das Formular verlangt eine fünfstellige Postleitzahl sowie eine nicht leere Straße und Stadt. Im letzten Schritt werden Lieferadresse, Zahlungsart und Gesamtbetrag vor dem Absenden nochmals angezeigt.

Die Auswahl zwischen Rechnung und Karte ist in der aktuellen Laborversion eine Simulation. Es werden keine Kartendaten erfasst und kein externer Zahlungsanbieter aufgerufen. An das Backend wird beim Abschluss nur die Lieferadresse übertragen. Diese Abgrenzung wurde gewählt, weil eine echte Zahlungsabwicklung einen externen Anbieter, zusätzliche Verträge und weitergehende Sicherheitsmaßnahmen benötigt. Nach erfolgreichem Abschluss zeigt das Frontend Bestellnummer, Status und Summe an und verlinkt zur Bestellhistorie.

Zu den behandelten Fehlerfällen gehören ein leerer Warenkorb, eine abgelaufene Sitzung, ungültige Adressdaten, ein nicht mehr in ausreichender Menge verfügbares Produkt und ein nicht erreichbarer Server. Solche Fälle werden im jeweiligen Arbeitsschritt angezeigt. Die bereits eingegebenen Formulardaten bleiben dabei erhalten, damit der Nutzer nicht von vorn beginnen muss.

Der gefüllte Warenkorb und die abschließende Prüfung im Checkout sind in Abbildung 5.2 dargestellt. Für den Nachweis wurden neutrale Demo-Adressdaten verwendet.

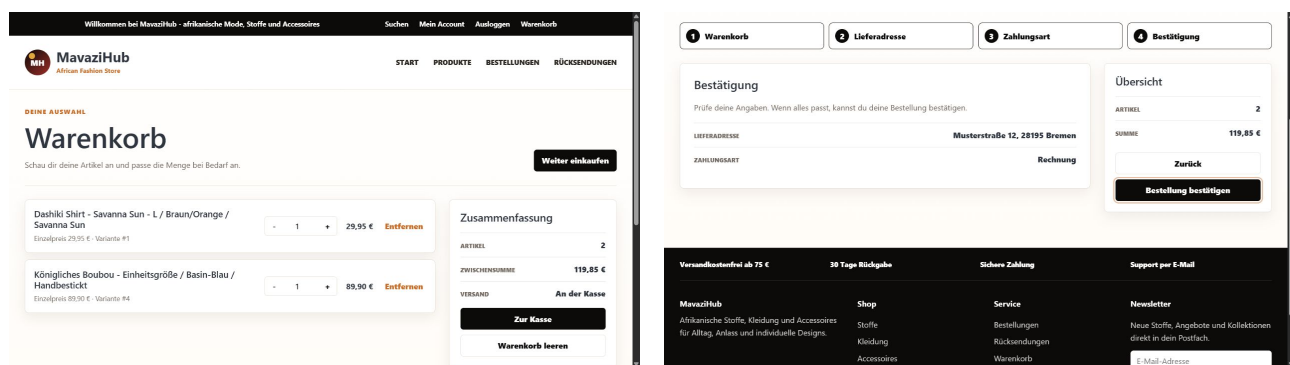


Abbildung 5.2.: Gefüllter Warenkorb und Bestätigungsschritt im Checkout

5.1.6. Bestellungen und Rücksendungen

Nach einer Bestellung kann der Nutzer seine eigenen Bestellungen in einer Übersicht aufrufen. Jede Karte zeigt eine verkürzte Bestellnummer, Datum, Anzahl der Positionen, Gesamtbetrag und aktuellen Status. Über die Detailansicht werden die vollständige Bestellnummer und alle bestellten Artikel mit Menge und Preis angezeigt.

Für den Bearbeitungsstand wird die wiederverwendbare Komponente **StatusTimeline** eingesetzt. Die normale Reihenfolge lautet:

Bestellt → Beahlt → In Bearbeitung → Versendet → Geliefert.

Der aktuelle und die bereits erreichten Schritte werden optisch markiert. Ein stornierter Auftrag wird zusätzlich mit seinem eigenen Status gekennzeichnet. Eine Rücksendung kann aus einer vorhandenen Bestellung gestartet werden. Der Nutzer wählt zuerst die Bestellung und anschließend die betroffenen Positionen. Für jeden Artikel kann höchstens die ursprünglich bestellte Menge eingetragen werden. Ein Grund kann freiwillig ergänzt werden. Ohne mindestens einen ausgewählten Artikel wird das Formular nicht abgesendet. Nach erfolgreicher Anfrage erscheint eine Bestätigung mit Rücksendenummer, Status und Artikelanzahl.

Auch Rücksendungen besitzen eine Status-Timeline. Der reguläre Weg lautet “Beantragt”, “In Prüfung”, “Genehmigt” und “Erstattet”. Für eine Ablehnung wird ein eigener Verlauf bis “Abgelehnt” dargestellt. Die Retourenübersicht verbindet jede Rücksendung mit der zugehörigen Bestellung und bietet einen direkten Link zu deren Details. Die Regeln für Statuswechsel werden im Backend-Kapitel erläutert.

Abbildung 5.3 zeigt Historie und Detailansicht mit Status-Timeline.

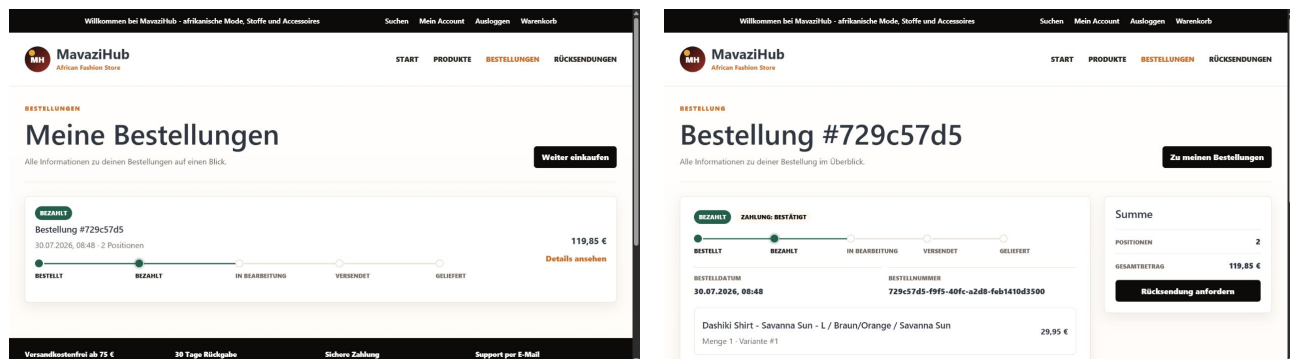


Abbildung 5.3.: Bestellhistorie und Bestelldetail mit Status-Timeline

Abbildung 5.4 belegt Artikelauswahl und angelegten Rücksendevorgang.

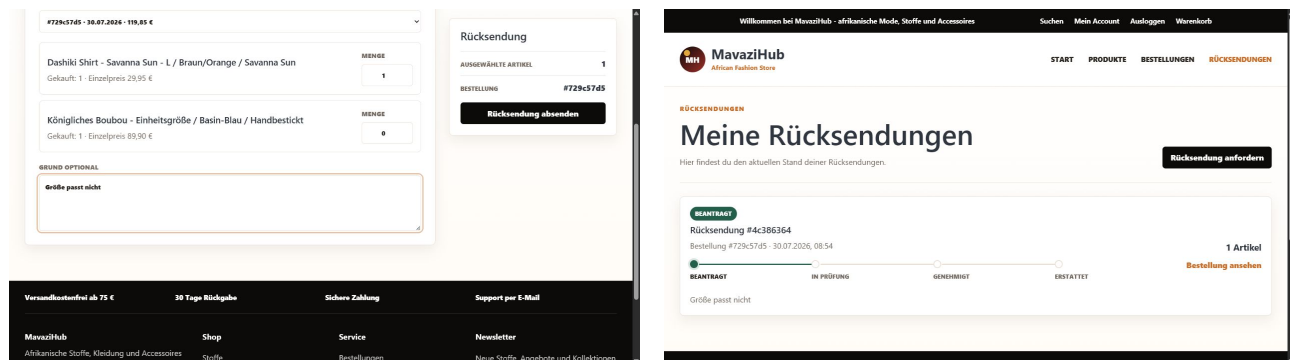


Abbildung 5.4.: Auswahl der Rücksendeartikel und Retourenübersicht

5.1.7. Admin-Frontend

Der Verwaltungsbereich verwendet mit dem `AdminLayout` ein eigenes Arbeitslayout. Eine feste Navigation führt zu Dashboard, Produkten, Kategorien, Bestellungen und Retouren. Die Bezeichnung richtet sich nach der Rolle; nur Administratoren sehen die Nutzerverwaltung.

Das Dashboard fasst vier Kennzahlen zusammen: offene Bestellungen, offene Retouren, aktive Produkte mit niedrigem Bestand und deaktivierte Produkte. Die Werte werden aus den aktuellen Antworten der Bestell-, Retouren- und Produkt-API berechnet. Die einzelnen Kacheln verlinken direkt zum passenden Arbeitsbereich. Werden einzelne Datenquellen nicht geladen, bleiben die anderen Kennzahlen nutzbar und es erscheint ein Hinweis.

In der Produktverwaltung können Produkte angelegt, bearbeitet, deaktiviert und wieder veröffentlicht werden. Das Formular enthält Name, Beschreibung, Preis, Kategorie und Lagerbestand. Es unterstützt mehrere Bildadressen und einen lokalen Upload von mehreren JPG-, PNG-, WebP- oder GIF-Dateien. Die Dateien werden als `multipart/form-data` an das Backend gesendet. Nach dem Upload übernimmt das Formular die zurückgegebenen Bildadressen. Das erste Bild wird als Hauptbild verwendet, weitere Bilder erscheinen später in der Galerie der Produktdetailseite. Eine direkte Bildvorschau im Adminformular ist im aktuellen Stand noch nicht vorhanden; die erfolgreiche Übernahme wird durch eine Meldung und die ausgefüllten Adressfelder bestätigt.

Kategorien können angelegt und bearbeitet werden. Für Produkte lassen sich außerdem Varianten mit Größe, Farbe, Muster und eigenem Lagerbestand pflegen. Varianten können aktiviert oder deaktiviert werden. Der genaue Bestand ist nur im Arbeitsbereich sichtbar. Kunden erhalten dagegen nur die in Abschnitt 5.1.4 beschriebenen Verfügbarkeitstexte.

Mitarbeiter und Administratoren sehen Tabellen für Bestellungen und Retouren. Dort kann der jeweilige Bearbeitungsstatus gewählt und gespeichert werden. Nach einer erfolgreichen Änderung zeigt die Oberfläche eine Bestätigung. Fehler werden ebenfalls direkt oberhalb der Tabelle ausgegeben. Administratoren erhalten zusätzlich die Nutzerverwaltung. Dort können die Rollen `ROLE_USER`, `ROLE_EMPLOYEE` und `ROLE_ADMIN` zugeordnet sowie Nutzerkonten aktiviert oder deaktiviert werden. Ein deaktiviertes Konto kann sich anschließend nicht mehr anmelden; die eigentliche Durchsetzung dieser Regel erfolgt im Backend.

Das Dashboard in Abbildung 5.5 zeigt einen realen Demo-Zustand mit einer offenen Bestellung und einer offenen Retoure. Dadurch wird sichtbar, wie die Kennzahlen die laufenden Vorgänge zusammenfassen.

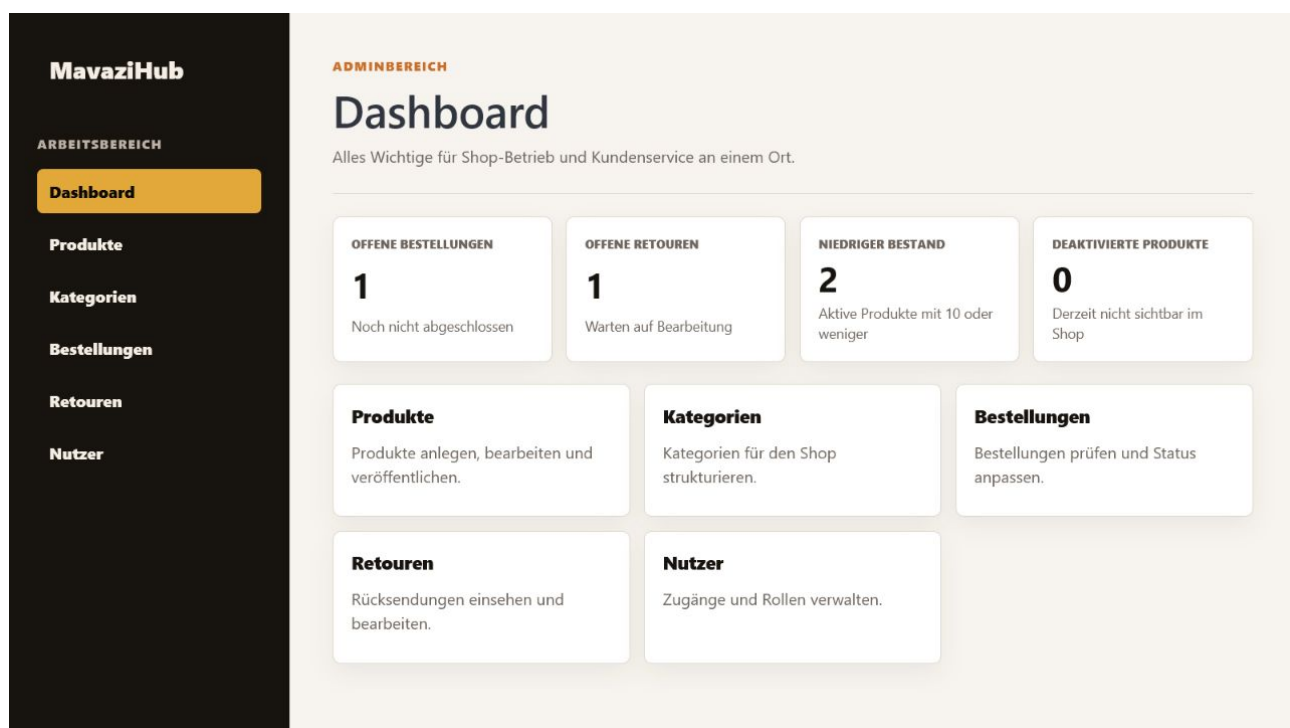


Abbildung 5.5.: Admin-Dashboard mit Kennzahlen aus dem Demo-Datenbestand

Abbildung 5.6 zeigt zwei weitere zentrale Verwaltungsfunktionen. Im Produktformular wurden zwei lokale Dateien hochgeladen und als Bildadressen übernommen. Die Nutzerverwaltung trennt den Demo-Admin und ein normales Kundenkonto anhand der zugewiesenen Rollen.

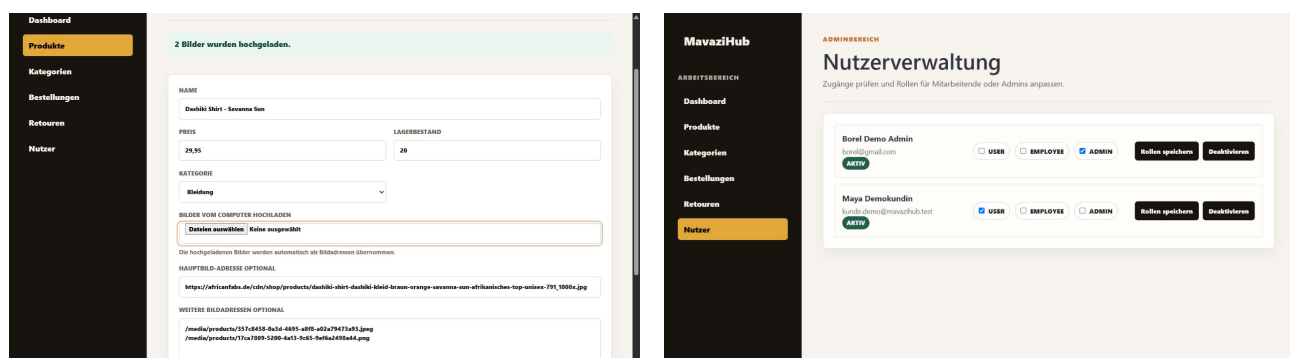


Abbildung 5.6.: Lokaler Produktbildupload und rollenbasierte Nutzerverwaltung

5.2. Backend

5.2.1. Datenbank und Migrationen

Für die persistente Speicherung verwendet MavaziHub eine relationale PostgreSQL-Datenbank. Im Docker-Compose-System wird dafür das Image `postgres:16-alpine` eingesetzt. Die Datenbank speichert die Daten der Benutzer- und Rollenverwaltung, des Produktkatalogs, des Warenkorbs, der Bestellungen sowie der Rücksendungen.

Das Datenbankschema wird nicht automatisch durch Hibernate erzeugt, sondern kontrolliert über Flyway-Migrationen aufgebaut. Die produktive Konfiguration verwendet deshalb die Einstellung `spring.jpa.hibernate.ddl-auto=validate`. Hibernate prüft beim Start, ob die implementierten JPA-Entities mit dem vorhandenen Schema übereinstimmen, nimmt jedoch selbst keine Schemaänderungen vor.

Flyway lädt die SQL-Dateien aus dem Verzeichnis `backend/src/main/resources/db/migration` und führt noch nicht angewendete Migrationen anhand ihrer Versionsnummer in aufsteigender Reihenfolge aus. Bereits ausgeführte Migrationen und ihre Prüfsummen werden in der technischen Tabelle `flyway_schema_history` dokumentiert. Dadurch ist der Aufbau des Datenbankschemas reproduzierbar und Änderungen bleiben nachvollziehbar.

Das Projekt enthält insgesamt 15 Migrationen. Tabelle 5.3 fasst diese kompakt nach fachlichen Bereichen zusammen.

Versionen	Inhalt
V1–V4	Aufbau der Benutzer- und Authentifizierungsverwaltung mit <code>roles</code> , <code>users</code> , <code>users_roles</code> und <code>refresh_tokens</code> .
V5–V8	Einführung von Warenkorb, Bestellungen und Rücksendungen. V8 gleicht die zunächst verwendeten UUID-Produktkennungen an die später eingeführten numerischen Produktkennungen an und ergänzt optionale Variantenreferenzen.
V9–V13	Aufbau des Produktkatalogs mit Kategorien, Produkten, Varianten und mehreren Produktbildern. Zusätzlich werden Demo-Daten sowie eine Eindeutigkeitsbedingung für Produktvarianten ergänzt.
V14–V15	Einrichten und Aktualisieren eines lokalen Demo-Administrators sowie Sicherstellen der benötigten Rollen und Rollenzuweisungen. Sensible Zugangsdaten werden nicht in der Dokumentation angegeben.

Tabelle 5.3.: Zusammenfassung der Flyway-Migrationen

Die Migrationen zeigen auch die schrittweise Entwicklung des Systems. So wurden Warenkorb und Bestellung zunächst unabhängig vom späteren Produktkatalog implementiert. Mit V8 wurden die entsprechenden Produktreferenzen angepasst. Danach führten V9 bis V13 das endgültige Katalogmodell mit Kategorien, Varianten und mehreren Produktbildern ein.

Im Docker-Compose-Betrieb wird ein benanntes Volume für PostgreSQL verwendet. Die gespeicherten Daten bleiben dadurch auch nach einem Neustart der Container erhalten. Der Backend-Container startet erst, nachdem der PostgreSQL-Healthcheck erfolgreich war. Anschließend führt Flyway die fehlenden Migrationen aus und Hibernate validiert das entstandene Schema.

Für automatisierte Backend-Tests wird statt PostgreSQL eine H2-In-Memory-Datenbank im PostgreSQL-Kompatibilitätsmodus verwendet. Dabei erzeugt Hibernate das Schema für jeden Testlauf mit `create-drop`; Flyway ist im Testprofil deaktiviert. Diese Konfiguration ermöglicht schnelle und isolierte Tests, ersetzt jedoch nicht den abschließenden Systemtest mit PostgreSQL und dem vollständigen Docker-Compose-Stack.

5.2.2. RESTAPI

Die REST-API ist unter dem Basispfad /api organisiert und in drei Sichtbarkeitsstufen unterteilt öffentlich (kein Login nötig), authentifiziert (jede eingeloggte Rolle) und administrativ (nur ROLE_ADMIN bzw. ROLE_ADMIN/ROLE_EMPLOYEE). Die Rechtevergabe erfolgt zentral in SecurityConfig, nicht in den einzelnen Controllern.

```
1 .authorizeHttpRequests(auth -> auth
2     //Deffentliche Endpoints
3     .requestMatchers("/api/health/**").permitAll()
4     .requestMatchers("/actuator/health/**").permitAll()
5     .requestMatchers(HttpMethod.GET, "/media/**").permitAll()
6     .requestMatchers("/swagger-ui/**", "/v3/api-docs/**", "/swagger-ui.
    html").permitAll()
7     .requestMatchers(HttpMethod.OPTIONS, "/*").permitAll()
8     .requestMatchers(HttpMethod.POST, "/api/auth/register", "/api/auth/
    login", "/api/auth/refresh", "/api/auth/logout").permitAll()
9     .requestMatchers(HttpMethod.GET, "/api/categories/**").permitAll()
10    .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
11    .requestMatchers("/api/cart/**").authenticated()
12
13    //Employee
14    .requestMatchers("/api/admin/categories/**").hasAnyAuthority("
    ROLE_ADMIN", "ROLE_EMPLOYEE")
15    .requestMatchers("/api/admin/products/**").hasAnyAuthority("
    ROLE_ADMIN", "ROLE_EMPLOYEE")
16    .requestMatchers("/api/admin/orders/**").hasAnyAuthority("ROLE_ADMIN
    ", "ROLE_EMPLOYEE")
17    .requestMatchers("/api/admin/returns/**").hasAnyAuthority("
    ROLE_ADMIN", "ROLE_EMPLOYEE")
18
19    //Admin
20    .requestMatchers("/api/admin/**").hasAuthority("ROLE_ADMIN")
21    .anyRequest().authenticated()
22 )
```

Quellcode 5.1: Configuration Spring Security

5.2.3. Interne Schnittstellen

Neben den REST-API bestehen folgende service-interne Abhängigkeiten (klien HTTP, direkt Java-AUfrufe via Dependency Injection). Diese Kopplungen zeugen, dass fachliche Konsistenz nicht auf Datenbankebene über Constraints, sondern explizit in der Service-Schicht sichergestellt wird.

Aufrufer	aufgerufener Service	Zweck
CartService	ProductService, ProductVariantService	aktives Produkt laden, Lagerbestand prüfen
OrderService	ProductService, ProductVariantService	Bestand beim Checkout reduzieren
OrderService	CartItemRepository	Warenkorb laden und nach Bestellung leeren
ReturnRequestService	OrderRepository	Prüfen, ob Bestellung existiert und dem Kunden gehört
ReturnRequestService	ReturnItemRepository	Bereits zurückgesendete Menge je Bestellposition
OrderHistoryService	ReturnItemRepository	Rückgabefähige Menge je Position berechnen
AuthServiceImpl	RefreshTokenServiceImpl, JwtService	Tokens bei Login/Refresh/Logout
JwtAuthenticationFilter	JwtService, CustomerDetailsService, AuthCookieService	Access-Token prüfen
ProductVariantService	ProductService	Prüfen, ob zugehöriges Produkt existiert

Tabelle 5.4.: Interne Beziehungen zwischen Klassen

5.2.4. Authentifizierung und Autorisierung

MavaziHub setzt für die Authentifizierung auf JSON Web Tokens (JWT), die in `httpOnly`-Cookies gespeichert werden. Beim Login prüft `AuthServiceImpl` E-Mail und Passwort über den `AuthenticationManager` von Spring Security; bei Erfolg erzeugt `JwtService` einen signierten Access-Token und `RefreshTokenServiceImpl` zusätzlich einen Refresh-Token in der Datenbank. `AuthCookieService` setzt beide sowie ein separates CSRF-Token als Cookies.

Bei jedem weiteren Request prüft der `JwtAuthenticationFilter`, ob ein gültiger Access-Token vorliegt, lädt den zugehörigen Nutzer frisch aus der Datenbank (statt sich nur auf den Token-Inhalt zu verlassen) und stellt so sicher, dass eine nachträgliche Sperrung sofort wirkt. Läuft der Access-Token ab, holt sich das Frontend automatisch über `POST /api/auth/refresh` ein neues Token-Paar, solange der Refresh-Token noch gültig ist. Beim Logout wird der Refresh-Token serverseitig widerrufen und alle Cookies werden gelöscht.

Die Autorisierung erfolgt rollenbasiert und zentral in `SecurityConfig`, nicht verteilt in den einzelnen Controllern. Anhand von URL-Mustern wird festgelegt, welche Endpunkte öffentlich sind (z. B. Produktkatalog, Login), welche jede eingeloggte Rolle erfordern (z. B. Warenkorb) und welche zusätzlich `ROLE_EMPLOYEE` bzw. ausschließlich `ROLE_ADMIN` voraussetzen (Produkt-, Bestell- und Rücksendungsverwaltung bzw. Nutzerverwaltung). Die Rollen selbst sind über das Enum `RoleName` sowie eine Many-to-Many-Beziehung zwischen `User` und `Role` im Datenmodell abgebildet.

Zusätzlich schützt der `CsrfValidationFilter` alle zustandsändernden Requests über ein Double-Submit-Cookie-Pattern: Der Wert des CSRF-Cookies muss mit einem vom Frontend mitgesendeten Header übereinstimmen, da `httpOnly`-Cookies vom Browser sonst automatisch auch bei ungewollten Cross-Site-Requests mitgeschickt würden.

5.2.5. Containerisierung und Systemstart

Das gesamte System wird über eine einzige `docker-compose.yml` mit drei Services orchestriert.

postgres-Service: Nutzt das offizielle Image *postgres:16-alpine*. Datenbankname, Benutzer und Passwort werden über Umgebungsvariablen mit sinnvollen Default-Werten gesetzt, sodass der Service auch ohne gesetzte `.env`-Datei startfähig ist. Ein Healthcheck über *pg_isready* prüft alle 10 Sekunden (5 Versuche, 5 Sekunden Timeout), ob die Datenbank bereit ist. Die Daten werden im benannten Volume *mavazihub_postgres_data* persistiert.

backend-Service: Wird aus dem lokalen Kontext `./backend` gebaut. Der Build erfolgt als Multi-Stage-Dockerfile: In Stage 1 (*maven:3.9.11-eclipse-temurin-17*) wird zunächst nur die `pom.xml` kopiert und *dependency:go-offline* ausgeführt, um Docker-Layer-Caching für Abhängigkeiten zu nutzen; anschließend wird der Quellcode kopiert und mit `mvn package` (ohne Tests) das JAR gebaut. In Stage 2 wird nur das schlanke *eclipse-temurin:17-jre*-Image verwendet, in das ausschließlich das fertige JAR aus Stage 1 kopiert wird – das finale Image enthält kein Maven und keinen Quellcode. Der Service erhält Datenbankzugangsdaten, den JWT-Secret-Key und das Media-Verzeichnis über Umgebungsvariablen und startet erst, wenn PostgreSQL als „healthy“ gemeldet wird. Produktbilder werden über das Volume *mavazihub_media* persistiert.

frontend-Service: Wird aus `./frontend/frontend` gebaut, ebenfalls als Multi-Stage-Build. In Stage 1 (*node:22-alpine*) werden Abhängigkeiten installiert (`npm ci`), der Build-Argument `VITE_API_BASE_URL` auf `/api` gesetzt und die Anwendung mit `npm run build` als statische Dateien nach `dist/` gebaut. In Stage 2 (**nginx:1.27-alpine**) wird die eigene `nginx.conf` nach `/etc/nginx/conf.d/default.conf` kopiert und die gebauten Dateien werden nach `/usr/share/nginx/html` übernommen. Der Service hängt vom Backend ab (`depends_on: backend`).

Nginx-Konfiguration: Nginx liefert die React-Anwendung aus (root `/usr/share/nginx/html`, SPA-Routing über `try_files uri /index.html`, damit serverseitige Routen auch bei direktem Aufruf funktionieren) und leitet alle Anfragen unter `/api/` sowie `/media/` per `proxy_pass` an `http://backend:8080` weiter. Dabei werden Standard-Proxy-Header gesetzt (Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto), damit das Backend die ursprüngliche Client-IP und das verwendete Protokoll kennt. `client_max_body_size 24m` erlaubt größere Uploads (z. B. Produktbilder) und ist auf das Multipart-Limit des Backends (`max-request-size: 24MB` in `application.yaml`) abgestimmt.

Netzwerk und Volumes: Alle drei Services sind über das Bridge-Netzwerk **mavazihub-network** verbunden, wodurch sie sich gegenseitig über ihre Servicenamen erreichen können. Zwei benannte Volumes (*mavazihub_postgres_data*, *mavazihub_media*) sorgen für Datenpersistenz über `docker compose down` und `docker compose up` hinweg – ohne diese Volumes würden Datenbankinhalte und hochgeladene Bilder bei jedem Neustart verloren gehen.

6. Verifikation und Tests

Die Verifikation von MavaziHub erfolgte auf mehreren Ebenen. Automatisierte Backend- und Frontend-Tests prüfen einzelne Komponenten und fachliche Regeln. Ergänzend wurde die vollständig containerisierte Anwendung als Gesamtsystem gestartet und der zentrale Kundenprozess manuell über die REST-Schnittstellen und die Benutzeroberfläche geprüft.

6.1. Teststrategie und Testumgebung

Die Backend-Tests wurden mit JUnit, Mockito, MockMvc und den Spring-Security-Testwerkzeugen umgesetzt. Insgesamt enthält der aktuelle Projektstand 33 automatisierte Testfälle. Diese prüfen insbesondere die Authentifizierung, die Cookie- und CSRF-Verarbeitung, den Warenkorb, die Bestellhistorie, Rücksendungen, Produktbilder sowie die rollenbasierte Zugriffskontrolle.

Für die Backend-Tests wird eine H2-In-Memory-Datenbank im PostgreSQL-Kompatibilitätsmodus verwendet. Hibernate erzeugt das Testschema für jeden Testlauf neu und entfernt es anschließend wieder. Flyway ist im Testprofil deaktiviert. Dadurch können die Tests unabhängig von einer lokal installierten PostgreSQL-Datenbank ausgeführt werden.

Im Frontend werden Vitest und die React Testing Library eingesetzt. Die 16 Testfälle konzentrieren sich auf sicherheits- und navigationsrelevante Funktionen. Dazu gehören das Übertragen des CSRF-Tokens, die automatische Erneuerung einer Sitzung nach einer HTTP-401-Antwort, die Verwaltung des angemeldeten Benutzers im `AuthContext` sowie die Weiterleitung durch geschützte Routen.

Zusätzlich wurde ein Systemtest mit dem vollständigen Docker-Compose-Stack durchgeführt. Dieser umfasst die drei Services PostgreSQL, Spring-Boot-Backend und React-Frontend. Nach dem erfolgreichen Start wurden die Health-Endpunkte des Backends und der Datenbank sowie der zentrale Kundenprozess von der Registrierung bis zur Rücksendung geprüft.

6.2. Testergebnisse

Tabelle 6.1 fasst die automatisierten Prüfungen und den Aufbau des Gesamtsystems zusammen.

Prüfung	Ausführung	Ergebnis
Backend-Tests	<code>mvn -q test</code>	Erfolgreich; 33 Testfälle ohne fehlgeschlagenen Test
Frontend-Tests	<code>npm run test</code>	Erfolgreich; 16 von 16 Testfällen bestanden
Frontend-Build	<code>npm run build</code>	TypeScript-Prüfung und Vite-Produktionsbuild erfolgreich
Code-Prüfung	<code>npm run lint</code>	Ohne gemeldete ESLint-Fehler abgeschlossen
Docker-Compose-Start	<code>docker compose up -build -d</code>	PostgreSQL, Backend und Frontend erfolgreich gestartet
Healthchecks	<code>/api/health</code> und <code>/api/health/db</code>	Anwendung und PostgreSQL meldeten jeweils den Status UP

Tabelle 6.1.: Ergebnisse der automatisierten Prüfungen

Neben den automatisierten Tests wurde der vollständige Kundenprozess mit der laufenden Docker-Compose-Anwendung geprüft. Tabelle 6.2 zeigt die dabei beobachteten HTTP-Ergebnisse.

Prüfschritt	HTTP-Status	Ergebnis
Neues Kundenkonto registrieren	201	Erfolgreich
Mit dem Kundenkonto anmelden	204	Erfolgreich
Warenkorb laden und bearbeiten	200	Erfolgreich
Bestellung über den Checkout aufgeben	201	Erfolgreich
Eigene Bestellhistorie laden	200	Erfolgreich
Details einer eigenen Bestellung laden	200	Erfolgreich
Rücksendbare Bestellpositionen ermitteln	200	Erfolgreich
Rücksendungsanfrage erstellen	201	Erfolgreich
Eigene Rücksendungsanfragen anzeigen	200	Erfolgreich

Tabelle 6.2.: Abnahmetest des zentralen Kundenprozesses

Die Ergebnisse zeigen, dass der für Version 1.0 vorgesehene Kernprozess funktioniert. Ein Kunde kann sich registrieren und anmelden, Produkte über den Warenkorb bestellen, die eigene Bestellung einsehen und anschließend eine Rücksendung anfordern. Die Backend-Tests ergänzen diesen positiven Ablauf um Fehler- und Sicherheitsfälle. Dazu gehören beispielsweise ungültige Mengen, fremde Bestellungen, unzulässige Rücksendemengen, deaktivierte Benutzerkonten und Zugriffe mit einer nicht ausreichenden Rolle.

Die Ergebnisse der automatisierten und manuellen Prüfungen werden im folgenden Abschnitt den Anforderungen des Pflichtenhefts zugeordnet. Zugangsdaten, Tokens und andere sensible Konfigurationswerte werden nicht dokumentiert.

6.3. Verifikationsmatrix

Die Verifikationsmatrix ordnet die Anforderungen des Pflichtenhefts dem tatsächlichen Implementierungsstand und den durchgeführten Nachweisen zu. Anforderungen mit demselben Status und demselben Nachweis werden aus Gründen der Übersichtlichkeit zusammengefasst. Die vollständigen Anforderungsbeschreibungen befinden sich im Pflichtenheft.

Als Nachweismethoden werden **Test (T)**, **Inspektion (I)** und **Review of Design (RoD)** verwendet. Ein Test bezeichnet einen automatisierten Testfall oder einen manuellen Abnahmeschritt. Bei einer Inspektion werden Quellcode, Konfiguration oder Datenbankschema geprüft. Ein Review of Design bewertet die Umsetzung anhand der Architektur- und Entwurfsdokumentation.

Die Status werden wie folgt verwendet:

- **Erfüllt:** vollständig umgesetzt und erfolgreich nachgewiesen,
- **Teilweise erfüllt:** grundsätzlich umgesetzt, jedoch mit einer dokumentierten Abweichung,
- **Nicht erfüllt:** im aktuellen Stand nicht umgesetzt,
- **Nicht vollständig verifiziert:** umgesetzt, aber noch nicht mit der im Pflichtenheft geforderten Messung oder Prüftiefe nachgewiesen.

Anforderung	Status	Nachweis	Ergebnis und Anmerkung
Kundenkonto und Authentifizierung			
FA-01, FA-03–FA-10	Erfüllt	T: Registrierung und Login im Systemtest; AuthControllerTest, AuthServiceImplTest; Frontend-Tests für Auth-Context und ProtectedRoute	Registrierung, eindeutige E-Mail-Adressen, Login, Logout, Rollenunterscheidung, geschützte Routen, Profilanzeige, Profilbearbeitung und Passwortänderung sind umgesetzt.
FA-02	Teilweise erfüllt	T/I: Registrierung und RegisterRequest geprüft	Name, E-Mail-Adresse, Telefonnummer und Passwort werden bei der Registrierung erfasst. Die Lieferadresse wird erst beim Checkout eingegeben und nicht im Kundenkonto gespeichert.
FA-11	Nicht erfüllt	I: Frontend-Routen und Auth-Endpunkte geprüft	Eine Funktion zum Zurücksetzen eines vergessenen Passworts ist im aktuellen Stand nicht implementiert.
Einkauf, Warenkorb und Bestellung			
FA-12–FA-25	Erfüllt	T: CartServiceTest und OrderServiceTest; manueller Kundenprozess vom Produktkatalog bis Checkout	Produktübersicht, Detailansicht, Suche, Filter, Varianten, Warenkorb, Mengenänderung, Lieferadresse, simulierte Zahlungsart, Bestellabschluss, Bestätigung, Kundenzuordnung und Leeren des Warenkorbs wurden erfolgreich geprüft.
Bestellhistorie und Rücksendungen			
FA-26–FA-35	Erfüllt	T: OrderHistoryServiceTest, OrderHistoryControllerTest, ReturnRequestServiceTest, MyReturnRequestControllerTest; manueller Abnahmetest	Kunden können ausschließlich eigene Bestellungen und Details anzeigen, Rücksendeartikel und Mengen auswählen, einen Grund angeben und Rücksendungen anlegen. Kunden und Mitarbeiter können die jeweils vorgesehenen Statusinformationen verwenden.
Produktverwaltung			
FA-36–FA-46	Erfüllt	T/I: Adminbereich manuell geprüft; ProductImageStorageServiceTest; REST-Controller und SecurityConfig inspiziert	Produktliste, Anlegen, Bearbeiten, Aktivieren und Deaktivieren, Lagerbestand, Bilder, Varianten und Kategorien sind im Verwaltungsbereich umgesetzt. Deaktivierte Produkte werden im öffentlichen Katalog nicht ausgegeben.
Mitarbeiter- und Rollenverwaltung			
FA-47–FA-50, FA-54–FA-55	Erfüllt	T: AdminSecurityIntegrationTest; manueller Test des Mitarbeiter- und Adminbereichs	Mitarbeiter können sich anmelden, abmelden und die allgemeinen Kontofunktionen verwenden. Administratoren können Rollen ändern sowie Konten aktivieren und deaktivieren. Deaktivierte Konten werden bei der Anmeldung abgelehnt.
FA-51	Teilweise erfüllt	I/T: AdminUsersPage und AdminUserController geprüft	Ein Administrator kann ein bereits registriertes Kundenkonto zur Mitarbeiterrolle hochstufen. Das direkte Anlegen eines neuen Mitarbeiterkontos im Adminbereich ist nicht umgesetzt.
FA-52–FA-53	Nicht erfüllt	I: Benutzer- und Authentifizierungsfunktionen geprüft	Einmalige Startpasswörter und ein gesonderter Prozess zur erstmaligen Aktivierung eines Mitarbeiterkontos sind nicht implementiert.
Diagnose und Testunterstützung			
FA-T01–FA-T02	Erfüllt	T/I: /api/health, /api/health/db und Flyway-Migrationen geprüft	Healthchecks für Backend und PostgreSQL melden den Status UP. Beispielkategorien und Beispielprodukte werden über versionierte Migrationen bereitgestellt.

Fortsetzung auf der nächsten Seite

Tabelle 6.3 – Fortsetzung

Anforderung	Status	Nachweis	Ergebnis und Anmerkung
FA-T03	Teilweise erfüllt	I: Migrationen und Rollenverwaltung geprüft	Ein lokaler Demo-Administrator wird vorbereitet. Kunden- und Mitarbeiterkonten können registriert und anschließend über Rollen konfiguriert werden, werden aber nicht vollständig als feste Testkonten bereitgestellt.
FA-T04	Erfüllt	T: Service-, Controller- und Frontend-Fehlertests; manueller Test ungültiger Eingaben	Ungültige Mengen, fremde Bestellungen, falsche Zugangsdaten und unzulässige Zugriffe werden abgelehnt und mit verständlichen Fehlermeldungen beantwortet.
FA-T05	Teilweise erfüllt	I: Backend-Konfiguration und Fehlerbehandlung geprüft	Spring Boot protokolliert Start- und Laufzeitfehler. Ein projektspezifisches, strukturiertes Fehlerprotokollierungskonzept wurde jedoch nicht zusätzlich umgesetzt.
Schnittstellenanforderungen			
INT-1	Nicht erfüllt	I: Docker-, Nginx- und Backend-Konfiguration geprüft	Die lokale Laborversion wird über HTTP betrieben. Eine HTTPS-Terminierung mit produktivem Zertifikat ist nicht Teil der aktuellen Ausbaustufe.
INT-2	Teilweise erfüllt	T/I: Authentifizierungs- und Axios-Tests; Cookie- und JWT-Konfiguration geprüft	Das Sitzungsmanagement basiert auf JWTs. Diese werden aus Sicherheitsgründen in httpOnly-Cookies statt im Authorization-Header übertragen. Damit besteht eine bewusste Abweichung von der Formulierung des Pflichtenhefts.
EXT-1	Nicht erfüllt	I/T: Checkout und Bestelldaten geprüft	Es ist kein externer Zahlungsanbieter integriert. Die Zahlungsart und der Zahlungsstatus werden in Version 1.0 ausschließlich simuliert; Kreditkartendaten werden nicht verarbeitet.
EXT-2	Nicht erfüllt	I: Backend-Abhängigkeiten und Bestellprozess geprüft	Eine SMTP- oder API-Anbindung für automatische E-Mails ist nicht implementiert. Die Bestellbestätigung wird unmittelbar in der Benutzeroberfläche angezeigt.
UI-1–UI-2	Erfüllt	T/I: Storefront und rollenbasierter Adminbereich manuell geprüft	Ein responsiver öffentlicher Shop und ein eigener rollenabhängiger Verwaltungsbereich sind vorhanden.
Bedienbarkeit und Operationalität			
BE/OP-1–BE/OP-2	Erfüllt	T: Checkout und Formulare manuell geprüft	Der Checkout umfasst höchstens vier Schritte. Ungültige Formularangaben werden verhindert und mit verständlichen Hinweisen angezeigt.
BE/OP-3	Nicht erfüllt	I: Adminseiten geprüft	Verwaltungsobjekte werden einzeln bearbeitet. Eine Mehrfachauswahl zur gleichzeitigen Bearbeitung mehrerer Bestellungen ist nicht implementiert.
BE/OP-4–BE/OP-5	Teilweise erfüllt	I/T: Tastaturbedienung, Fokusdarstellung, Alternativtexte und Statusmeldungen geprüft	Grundlegende Tastaturnavigation, sichtbarer Fokus, Alternativtexte und semantische Statusmeldungen sind vorhanden. Eine vollständige WCAG-2.1-AA- oder Screenreader-Abnahme wurde nicht durchgeführt.
Qualitätsanforderungen			
QA-01	Erfüllt	T: <code>docker compose up -build -d</code> ; Healthchecks und Erreichbarkeit geprüft	PostgreSQL, Backend und Frontend konnten gemeinsam als Docker-Compose-System gestartet werden.

Fortsetzung auf der nächsten Seite

Tabelle 6.3 – Fortsetzung

Anforderung	Status	Nachweis	Ergebnis und Anmerkung
QA-02	Erfüllt	T/I: PostgreSQL-Volumes, Bestell- und Rücksendungsdaten geprüft	Bestellungen und Rücksendungen werden in PostgreSQL gespeichert. Benannte Docker-Volumes sichern die Daten über einen Container-Neustart hinaus.
QA-03	Erfüllt	T: Service- und Controller-Tests für fehlerhafte Eingaben	Fehlerhafte Aktionen werden vor der Speicherung abgebrochen. Tests prüfen unter anderem ungültige Mengen, unberechtigte Zugriffe und inkonsistente Rücksendungen.
QA-04–QA-05	Nicht vollständig verifiziert	T: funktionale Systemtests ohne formale Zeitmessung	Die Abläufe funktionierten unter den verwendeten Laborbedingungen ohne auffällige Verzögerungen. Die geforderten Zwei- beziehungsweise Drei-Sekunden-Grenzen wurden jedoch nicht durch einen reproduzierbaren Performance-Test gemessen.
QA-06–QA-09	Erfüllt	T/I: ProtectedRoute-Tests, AdminSecurityIntegration-Test, Bestell- und Rücksendungstests, SecurityConfig und Passwortspeicherung geprüft	Geschützte Funktionen verlangen Authentifizierung. Rollen werden serverseitig geprüft, Passwörter als BCrypt-Hash gespeichert und Kunden können nur eigene Bestellungen und Rücksendungen laden.
QA-10	Nicht vollständig verifiziert	T: manueller Browser- und Responsive-Test	Die Anwendung wurde im primären Entwicklungsbrowser geprüft. Eine dokumentierte vollständige Abnahme in aktuellen Versionen von Chrome, Firefox und Edge liegt nicht vor.
QA-11	Erfüllt	T/I: Docker-Compose-Start und Projektkonfiguration geprüft	Frontend, Backend und PostgreSQL können gemeinsam gestartet werden, ohne die Systemkomponenten einzeln auf dem Rechner installieren und konfigurieren zu müssen.

Tabelle 6.3.: Verifikationsmatrix für MavaziHub

Die Matrix zeigt, dass der Kernprozess der Version 1.0 von der Registrierung über Warenkorb und Bestellung bis zur Bestellhistorie und Rücksendung vollständig umgesetzt und geprüft wurde. Abweichungen bestehen insbesondere bei externen Integrationen, HTTPS, der direkten Erstellung neuer Mitarbeiterkonten sowie bei formalen Performance- und Barrierefreiheitstests. Diese Punkte werden im folgenden Fazit als Grenzen des aktuellen Projektstands und als mögliche Erweiterungen eingeordnet.

7. Fazit und Ausblick

7.1. Zusammenfassung und Erfüllungsgrad

Mit MavaziHub wurde ein funktionsfähiger Prototyp einer B2C-E-Commerce-Plattform für die Tamu Fashion GmbH entwickelt. Das System überführt den zuvor überwiegend informellen Verkaufsprozess in einen strukturierten digitalen Ablauf. Gäste können den Produktkatalog durchsuchen und Produktdetails aufrufen. Registrierte Kunden können Produkte in den Warenkorb legen, eine Bestellung aufgeben, ihre Bestellhistorie einsehen und Rücksendungen anfordern. Für Mitarbeiter und Administratoren stehen zusätzlich rollenabhängige Verwaltungsfunktionen zur Verfügung.

Die Anwendung wurde als mehrschichtiges System umgesetzt. Das Frontend basiert auf React und TypeScript, während das Backend mit Spring Boot entwickelt wurde. Die Kommunikation erfolgt über eine REST-Schnittstelle. PostgreSQL übernimmt die persistente Speicherung der Anwendungsdaten und Flyway verwaltet die versionierte Entwicklung des Datenbankschemas. Frontend, Backend und Datenbank können gemeinsam über Docker Compose gestartet werden.

Die Mindestanforderungen des zentralen Kundenprozesses wurden weitgehend erfüllt. Registrierung, Anmeldung, Produktanzeige, Warenkorb, Checkout, Bestellhistorie und Rücksendung wurden im Gesamtsystem erfolgreich geprüft. Auch die rollenbasierte Zugriffskontrolle, die Produktverwaltung und die Bearbeitung von Bestellungen und Rücksendungen sind im aktuellen Implementierungsstand vorhanden.

Die technische Umsetzung wurde durch automatisierte Backend- und Frontend-Tests sowie einen manuellen System- und Abnahmetest verifiziert. Zusätzlich wurden der Produktionsbuild des Frontends, die Healthchecks des Backends und der Datenbank sowie der gemeinsame Start des Docker-Compose-Stacks erfolgreich geprüft. Die Verifikationsmatrix in Abschnitt 6.3 zeigt den Erfüllungsgrad der einzelnen Anforderungen und dokumentiert bestehende Abweichungen.

Die gewählten Architekturentscheidungen haben sich für den Projektumfang als zweckmäßig erwiesen. Die Trennung zwischen Frontend, Backend und Datenbank ermöglicht eine klare Aufteilung der Verantwortlichkeiten. DTOs, Services und Repositories begrenzen die Abhängigkeiten innerhalb des Backends. Die Verwendung von Flyway verhindert unkontrollierte Schemaänderungen und sorgt dafür, dass der Datenbankstand reproduzierbar aufgebaut werden kann. Durch Docker Compose lässt sich das Gesamtsystem außerdem auf unterschiedlichen Entwicklungsrechnern mit vergleichbarer Konfiguration starten.

MavaziHub ist damit als Labor- und Demonstrationssystem funktionsfähig. Die Anwendung stellt jedoch noch kein vollständig produktionsreifes Shopsystem dar. Insbesondere externe Dienste und einige nichtfunktionale Anforderungen wurden im vorgesehenen Projektzeitraum nur vereinfacht umgesetzt oder noch nicht formal geprüft.

7.2. Grenzen und mögliche Erweiterungen

Die Bezahlung wird im aktuellen Stand lediglich simuliert. Es erfolgt keine Anbindung an einen realen Zahlungsdienstleister und es werden keine Kreditkartendaten verarbeitet. Als spätere Erweiterung könnte beispielsweise ein Anbieter wie Stripe oder PayPal über eine klar abgegrenzte Zahlungsschnittstelle angebunden werden.

Bestell- und Statusinformationen werden derzeit ausschließlich innerhalb der Anwendung dargestellt. Eine automatische Benachrichtigung per E-Mail ist nicht implementiert. Die Integration eines Maildienstes könnte künftig Bestellbestätigungen, Versandinformationen, Passwortzurücksetzungen und Änderungen am Rücksendestatus ermöglichen.

Die lokale Ausführung verwendet HTTP. Für einen produktiven Betrieb wären eine HTTPS-Terminierung, sichere Umgebungsvariablen, ein separates Secret-Management sowie strengere Produktionskonfigurationen notwendig. Zusätzlich sollte die lokale Ablage von Produktbildern durch einen skalierbaren Objektspeicher oder einen Cloud-Speicherdienst ersetzt werden.

Die automatisierten Tests decken wichtige Geschäfts- und Sicherheitsregeln ab. Formale Last-, Performance-, Penetrations- und vollständige Barrierefreiheitstests wurden jedoch nicht durchgeführt. Vor einem produktiven Einsatz wären insbesondere reproduzierbare Antwortzeitmessungen, Browser-Kompatibilitätstests, Sicherheitsanalysen und eine Prüfung nach WCAG 2.1 erforderlich.

Weitere fachliche Erweiterungen wären eine direkte Mitarbeiteranlage durch Administratoren, ein Prozess für vergessene Passwörter, Versanddienstleister mit Sendungsverfolgung, Lagerwarnungen, Rabattaktionen sowie ein erweitertes Dashboard mit Verkaufs- und Retourenstatistiken.

Die vorhandene modulare Architektur bietet für diese Erweiterungen eine geeignete Grundlage. Neue Funktionen können durch zusätzliche Frontend-Seiten, REST-Endpunkte, Services und Flyway-Migrationen ergänzt werden, ohne den bestehenden Kernprozess grundlegend neu entwickeln zu müssen.

Literaturverzeichnis

A. Anhang

A.1. Architecture Decision Records

A.1.1. ADR-001 React/TypeScript als Frontend-Framework (SPA)

Status: accepted

Deciders: Team

Date: 2026-05-27

Technical Story Auswahl des Frontend-Frameworks für die MavaziHub-SPA

Context and Problem Statement Für die B2C-E-Commerce-Plattform MavaziHub wird ein Frontend-Framework benötigt, das eine klare Trennung von Backend und Oberfläche ermöglicht und vom vierköpfigen Studierendenteam innerhalb des vorgegebenen Projektzeitraums produktiv erlernt und eingesetzt werden kann. Welches Frontend-Framework soll für die Umsetzung der Single-Page-Application verwendet werden?

Decision Drivers

- Begrenzte Projektlaufzeit und feste Abgabefrist
- Unterschiedliches Vorwissen der Teammitglieder
- Notwendigkeit einer typsicheren Anbindung an die REST-API
- Angemessenheit des architektonischen Overheads für ein Laborprojekt dieser Größe

Considered Options

- Angular
- React mit TypeScript
- Server-seitig gerenderte Templates (z. B. Thymeleaf)

Decision Outcome **Chosen option:** React mit TypeScript, weil Angular nach Recherche und Teamdiskussion als Framework eingeschätzt wurde, das primär auf große, langfristig skalierende Enterprise-Projekte mit entsprechend striktem Strukturüberbau ausgelegt ist. React ermöglichte dem Team dagegen eine deutlich flachere Lernkurve und damit eine schnellere produktive Umsetzung der geforderten Kernfunktionen innerhalb der Projektlaufzeit.

Positive Consequences

- Schnellere Einarbeitung des Teams, mehr Zeit für fachliche Umsetzung statt Framework-Lernaufwand
- Klare Trennung von Frontend und Backend, unabhängige Weiterentwicklung beider Seiten möglich
- Typsichere API-Anbindung durch TypeScript reduziert Laufzeitfehler

Negative Consequences

- Frontend und Backend müssen getrennt gebaut und deployed werden (zwei Docker-Images)
- CORS-Konfiguration im Backend notwendig, insbesondere für die lokale Entwicklung ohne Nginx-Proxy

Pros and Cons of the Options

Angular Vollwertiges Framework mit eigenem CLI, striktem Strukturkonzept und integriertem Dependency-Injection-System, primär für große Enterprise-Anwendungen konzipiert.

- Gut, weil es eine sehr klare, vorgegebene Projektstruktur mitbringt
- Gut, weil es für sehr große Teams und langlebige Projekte gut skaliert
- Nicht gut, weil die Einstiegshürde für das Team deutlich höher lag als bei React
- Nicht gut, weil der Strukturüberbau in keinem Verhältnis zum Umfang eines vierköpfigen Laborprojekts steht

React mit TypeScript Bibliothek zum Bau von Nutzeroberflächen mit großem Ökosystem und moderater Lernkurve.

- Gut, weil die Lernkurve für das Team spürbar flacher war
- Gut, weil eine sehr große Community und viele Beispielprojekte die Einarbeitung erleichtern
- Gut, weil TypeScript zusätzliche Typsicherheit bei der API-Anbindung liefert
- Nicht gut, weil im Vergleich zu Angular weniger feste Konventionen vorgegeben sind und das Team eigene Strukturentscheidungen treffen musste

Server-seitig gerenderte Templates (Thymeleaf) Serverseitiges Rendering direkt aus dem Spring-Boot-Backend heraus, ohne separate SPA.

- Gut, weil kein separates Frontend-Deployment nötig wäre
- Gut, weil keine CORS-Problematik entsteht
- Nicht gut, weil keine klare Trennung von Präsentations- und Fachlogik
- Nicht gut, weil moderne, reaktive Nutzererfahrungen (z. B. Warenkorb-Updates ohne Seitenneuladen) deutlich aufwendiger umzusetzen wären

Market Adoption React zählt laut gängigen Entwicklerumfragen zu den am weitesten verbreiteten Frontend-Bibliotheken im Web-Umfeld und wird sowohl in kleinen als auch großen Projekten breit eingesetzt, was die Verfügbarkeit von Dokumentation, Tutorials und Drittanbieter-Bibliotheken (z. B. Axios, React Router) begünstigt.

Consequences Die Entscheidung für React zieht Folgeentscheidungen nach sich, u. a. zur CORS-Konfiguration, zur Nginx-Proxy-Lösung im containerisierten Betrieb (siehe ADR-006) sowie zur Struktur des `src`-Verzeichnisses (Pages, Components, API-Client), die im Kapitel 5.1 dokumentiert wird.

Links

- Related to ADR-006
-

A.1.2. ADR-002 Spring Boot als REST-API-Backend

Status: accepted

Deciders: Team

Date: 2026-05-27

Technical Story Auswahl des Backend-Frameworks für die MavaziHub-REST-API

Context and Problem Statement Das Backend muss eine REST-API mit Authentifizierung, Rollenmodell, relationaler Datenanbindung und Validierung bereitstellen. Welches Backend-Framework soll dafür eingesetzt werden?

Decision Drivers

- Vorwissen des Teams
- Ausgereiftes Ökosystem für Security, ORM und Validierung
- Gute Integrationsfähigkeit mit PostgreSQL und Docker
- Verfügbarkeit von Test-Werkzeugen (JUnit, MockMvc, Mockito)

Considered Options

- Spring Boot (Java)
- Node.js/Express (JavaScript/TypeScript)

Decision Outcome **Chosen option:** Spring Boot, weil das Team bereits Vorwissen in Java/Spring mitbrachte und Spring Boot ein ausgereiftes, gut dokumentiertes Ökosystem für Security (Spring Security), Datenzugriff (Spring Data JPA) und Validierung (Bean Validation) bietet, das die Anforderungen des Pflichtenhefts direkt abdeckt.

Positive Consequences

- Einheitliches, gut getestetes Security-Modell über Spring Security realisierbar
- Deklarative Validierung über Annotationen (@Valid, @NotBlank, @Email) reduziert Boilerplate-Code
- Gute Testunterstützung durch MockMvc und Mockito

Negative Consequences

- Java/Maven-Toolchain erforderlich, längere Build-Zeit im Docker-Image im Vergleich zu interpretierten Sprachen
- Höherer Ressourcenverbrauch der JVM im Vergleich zu leichtgewichtigeren Node.js-Laufzeitumgebungen

Pros and Cons of the Options

Spring Boot (Java)

- Gut, weil Team-Vorwissen direkt genutzt werden kann
- Gut, weil Spring Security eine ausgereifte, gut dokumentierte Sicherheitsinfrastruktur bietet
- Gut, weil Spring Data JPA den Datenzugriff stark vereinfacht
- Nicht gut, weil der Ressourcen- und Build-Zeit-Overhead größer ist als bei leichtgewichtigeren Alternativen

Node.js/Express

- Good, because dieselbe Sprache (JavaScript/TypeScript) wie im Frontend genutzt werden könnte
- Good, because die Laufzeitumgebung ressourcenschonender ist
- Nicht gut, weil dem Team weniger Vorwissen in diesem Ökosystem vorlag
- Nicht gut, weil Security- und ORM-Lösungen weniger einheitlich integriert sind als bei Spring

Market Adoption Spring Boot ist im Java-Enterprise-Umfeld eines der am weitesten verbreiteten Frameworks für REST-APIs und wird sowohl in Ausbildung als auch in produktiven Systemen breit eingesetzt.

Consequences Die Wahl von Spring Boot bedingt Folgeentscheidungen zu PostgreSQL als relationaler Datenbank (ADR-003), zu Flyway für Migrationen (ADR-004) sowie zur JWT-basierten Authentifizierung (ADR-007).

Links

- Related to ADR-003
 - Related to ADR-004
-

A.1.3. ADR-003 PostgreSQL als relationale Datenbank

Status: accepted

Deciders: Team

Date: 2026-05-27

Context and Problem Statement MavaziHub verwaltet stark strukturierte, relational verknüpfte Daten (Nutzer, Rollen, Produkte, Varianten, Warenkorb, Bestellungen, Rücksendungen). Welches Datenbanksystem soll dafür eingesetzt werden?

Decision Drivers

- Klare Fremdschlüsselbeziehungen zwischen Fachobjekten (z. B. OrderItem → Order, ReturnItem → OrderItem)
- Notwendigkeit von Transaktionssicherheit beim Checkout (Bestandsreduktion + Bestellanlage)
- Kompatibilität mit Spring Data JPA und Flyway

Considered Options

- PostgreSQL
- MongoDB (dokumentenorientiert)

Decision Outcome Chosen option: PostgreSQL, weil die Fachdaten von MavaziHub stark relational sind und transaktionale Konsistenz beim Bestellprozess (z. B. gleichzeitige Bestandsreduktion mehrerer Positionen) benötigt wird, was ein relationales System mit ACID-Garantien deutlich natürlicher abbildet als ein dokumentenorientiertes System.

Positive Consequences

- Fremdschlüsselbeziehungen und referenzielle Integrität werden von der Datenbank selbst unterstützt
- Transaktionale Konsistenz beim Checkout und bei Rücksendungen ist gewährleistet
- Direkte, ausgereifte Unterstützung durch Spring Data JPA und Flyway

Negative Consequences

- Schema muss vorab modelliert und bei Änderungen migriert werden
- Horizontale Skalierung ist aufwendiger als bei dokumentenorientierten Systemen

Pros and Cons of the Options

PostgreSQL

- Gut, weil es relationale Integrität und Transaktionssicherheit bietet
- Gut, weil es sich nahtlos mit Spring Data JPA und Flyway kombinieren lässt
- Nicht gut, weil Schemaänderungen explizit migriert werden müssen

MongoDB

- Gut, weil es flexible, schemalose Datenmodelle ermöglicht
- Nicht gut, weil referenzielle Integrität zwischen Fachobjekten manuell in der Anwendung sichergestellt werden müsste
- Nicht gut, weil transaktionale Mehrdokument-Operationen komplexer zu handhaben sind

Market Adoption PostgreSQL ist eine der am weitesten verbreiteten Open-Source-Datenbanken im relationalen Umfeld und wird sowohl im akademischen als auch im produktiven Kontext breit eingesetzt.

Consequences Die Entscheidung für PostgreSQL bedingt den Einsatz von Flyway zur Schemaversionierung (ADR-004) und von H2 als In-Memory-Alternative für automatisierte Tests.

Links

- Related to ADR-004
-

A.1.4. ADR-004 Flyway für Datenbank-Migrationen

Status: accepted

Deciders: Person 3, Person 4

Date: 2026-05-27

Context and Problem Statement Das Datenbankschema muss nachvollziehbar, versioniert und reproduzierbar aufgebaut werden können, insbesondere für den containerisierten Systemstart über Docker Compose. Wie soll das Schema verwaltet werden?

Decision Drivers

- Reproduzierbarkeit des Schemas bei jedem Neustart des Systems
- Vermeidung impliziter, unkontrollierter Schemaänderungen durch Hibernate im Betrieb
- Nachvollziehbarkeit der Schemaentwicklung über die Projektlaufzeit

Considered Options

- Flyway-Migrationen
- Hibernate `ddl-auto: update`
- Manuelles Schema-Management ohne Werkzeugunterstützung

Decision Outcome **Chosen option:** Flyway-Migrationen, weil versionierte SQL-Migrationsdateien eine nachvollziehbare und reproduzierbare Schemaentwicklung ermöglichen und verhindern, dass Hibernate im laufenden Betrieb unkontrolliert Schemaänderungen vornimmt.

Positive Consequences

- Schemaentwicklung ist über die Migrationshistorie (V1–V15) nachvollziehbar
- `ddl-auto: validate` stellt sicher, dass die Anwendung nur startet, wenn Entity-Mapping und tatsächliches Schema übereinstimmen
- Reproduzierbarer Systemstart in Docker Compose, da Flyway beim Backend-Start automatisch alle ausstehenden Migrationen anwendet

Negative Consequences

- Jede Schemaänderung erfordert eine zusätzliche Migrationsdatei, auch für kleine Anpassungen
- Bei fehlerhaften Migrationen ist manuelles Eingreifen nötig, da Flyway keine automatischen Rollbacks in der Community-Version unterstützt

Pros and Cons of the Options

Flyway-Migrationen

- Gut, weil Schemaänderungen versioniert und nachvollziehbar sind
- Gut, weil der Systemstart reproduzierbar bleibt
- Nicht gut, weil zusätzlicher Aufwand bei jeder noch so kleinen Schemaänderung entsteht

Hibernate ddl-auto: update

- Gut, weil keine manuellen Migrationsdateien nötig wären
- Nicht gut, weil Schemaänderungen implizit und schwer nachvollziehbar erfolgen
- Nicht gut, weil ungewollte Datenverluste bei automatischen Änderungen möglich sind

Manuelles Schema-Management

- Gut, weil volle Kontrolle über jede SQL-Anweisung besteht
- Nicht gut, weil keine Werkzeugunterstützung für Versionierung und Reproduzierbarkeit vorhanden ist
- Nicht gut, weil der Aufwand im Team unnötig hoch wäre

Market Adoption Flyway ist eines der etabliertesten Migrationswerkzeuge im Java/Spring-Ökosystem und wird häufig in Kombination mit Spring Boot eingesetzt.

Consequences Jede Schemaänderung im Projekt muss künftig als neue, fortlaufend nummerierte Migrationsdatei unter `db/migration` erfolgen; Hibernates automatische Schemaanpassung bleibt deaktiviert.

Links

- Related to ADR-003
-

A.1.5. ADR-005 Vollständiger Systembetrieb über Docker Compose

Status: accepted

Deciders: Team

Date: 2026-05-27

Context and Problem Statement Das System besteht aus mehreren Komponenten (Frontend, Backend, Datenbank), die reproduzierbar gestartet werden müssen – sowohl für die tägliche Entwicklung im Team als auch für die Abnahme/Demo. Wie soll der Gesamtbetrieb organisiert werden?

Decision Drivers

- Reproduzierbarer Start mit einem einzigen Befehl für Review und Abnahme
- Vermeidung von Abhängigkeiten von individuell unterschiedlich konfigurierten Entwicklerrechnern
- Einfache Handhabung im vierköpfigen Team

Considered Options

- Docker Compose mit drei Services (postgres, backend, frontend)
- Manuelles Aufsetzen jeder Komponente einzeln auf jedem Rechner

Decision Outcome Chosen option: Docker Compose mit drei Services, weil damit der gesamte Stack mit einem einzigen Befehl (`docker compose up -build -d`) reproduzierbar gestartet werden kann, unabhängig von lokal installierter Software außer Docker selbst.

Positive Consequences

- Ein-Befehl-Start für Entwicklung, Review und Abnahme
- Einheitliche Umgebung für alle Teammitglieder, unabhängig vom Betriebssystem
- Healthcheck- und Startabhängigkeiten (`depends_on: condition: service_healthy`) verhindern Startreihenfolge-Probleme

Negative Consequences

- Höherer initialer Konfigurationsaufwand (Dockerfiles, Compose-Datei, Netzwerkkonfiguration)
- Build-Zeit beim ersten Start, insbesondere für das Backend-Image

Pros and Cons of the Options

Docker Compose

- Good, because der komplette Stack reproduzierbar mit einem Befehl startet
- Good, because Healthchecks die korrekte Startreihenfolge erzwingen
- Bad, because initialer Konfigurationsaufwand höher ist als bei manuellem Setup

Manuelles Aufsetzen

- Gut, weil kein Docker-Wissen im Team notwendig wäre
- Nicht gut, weil jedes Teammitglied die Umgebung individuell und potenziell unterschiedlich konfiguriert
- Nicht gut, weil Reproduzierbarkeit für Review/Abnahme nicht gewährleistet ist

Market Adoption Docker Compose ist ein weit verbreitetes Standardwerkzeug zur Orchestrierung mehrerer Container in Entwicklungs- und kleineren Produktionsumgebungen.

Consequences Alle Umgebungsvariablen (z. B. `JWT_SECRET_KEY`, Datenbankzugangsdaten) werden über `.env`/Compose-Umgebungsvariablen konfiguriert; feste `container_name`-Werte werden bewusst vermieden, um Namenskonflikte bei parallelen Projektkopien zu verhindern.

Links

- Related to ADR-006
-

A.1.6. ADR-006 Nginx als Webserver und Reverse Proxy im Frontend-Container

Status: accepted

Deciders: Team

Date: 2026-05-27

Context and Problem Statement Der Browser darf aus Sicherheits- und Einfachheitsgründen nicht direkt mit dem Backend kommunizieren, und die React-SPA benötigt client-seitiges Routing auch bei direktem Aufruf einer Unterseite. Wie soll der Zugriff des Browsers auf Frontend und Backend organisiert werden?

Decision Drivers

- Einheitlicher Einstiegspunkt für den Browser
- Vermeidung von CORS-Problemen im containerisierten Betrieb
- Notwendigkeit von SPA-Routing-Unterstützung (`try_files`)

Considered Options

- Nginx als Reverse Proxy im Frontend-Container
- Direkter Browser-Zugriff auf das Backend mit CORS-Konfiguration

Decision Outcome **Chosen option:** Nginx als Reverse Proxy, weil dadurch ein einheitlicher Einstiegspunkt für den Browser entsteht, das Backend aus Nutzersicht unsichtbar bleibt und SPA-Routing zentral über `try_files $uri /index.html` gelöst werden kann.

Positive Consequences

- Keine CORS-Konfiguration im produktiven, containerisierten Betrieb notwendig, da Frontend und API unter demselben Origin erreichbar sind
- SPA-Routing funktioniert auch bei direktem Aufruf einer Unterseite
- Backend-Port muss aus Nutzersicht nicht öffentlich bekannt sein

Negative Consequences

- Zusätzlicher Netzwerk-Hop zwischen Nginx und Backend
- Proxy-Konfiguration (`nginx.conf`) muss gepflegt und bei neuen Backend-Pfaden erweitert werden

Pros and Cons of the Options

Nginx als Reverse Proxy

- Gut, weil Frontend und Backend unter demselben Origin erscheinen
- Gut, weil SPA-Routing zentral gelöst wird
- Nicht gut, weil eine zusätzliche Konfigurationsdatei gepflegt werden muss

Direkter Browser-Zugriff mit CORS

- Gut, weil kein zusätzlicher Proxy-Hop notwendig wäre
- Nicht gut, weil CORS-Konfiguration fehleranfällig und in jeder Umgebung neu abzustimmen ist
- Nicht gut, weil der Backend-Port direkt öffentlich erreichbar sein müsste

Market Adoption Nginx ist einer der verbreitetsten Webserver/Reverse-Proxys und wird häufig als Einstiegspunkt vor containerisierten Anwendungen eingesetzt.

Consequences Alle Anfragen unter `/api/` und `/media/` werden von Nginx an `http://backend:8080` weitergeleitet; die lokale Entwicklungsvariante nutzt stattdessen den Vite-Dev-Server-Proxy mit vergleichbarer Funktion.

Links

- Related to ADR-005
 - Related to ADR-001
-

A.1.7. ADR-007 JWT in httpOnly-Cookies statt im Browser-localStorage

Status: accepted

Deciders: Team

Date: 2026-07-27

Context and Problem Statement Nach erfolgreichem Login muss der Access-Token sicher im Browser gespeichert werden, sodass er bei jedem Request automatisch mitgesendet wird, ohne ein hohes Risiko für Tokendiebstahl über clientseitige Angriffe einzugehen. Wo soll der JWT gespeichert werden?

Decision Drivers

- Schutz vor XSS-basiertem Tokendiebstahl
- Automatisches Mitsenden des Tokens ohne manuelles Handling im Frontend-Code
- Kompatibilität mit dem zustandslosen REST-API-Ansatz

Considered Options

- httpOnly-Cookie
- Speicherung im Browser-localStorage

Decision Outcome Chosen option: httpOnly-Cookie, weil httpOnly-Cookies für JavaScript im Browser nicht auslesbar sind und damit deutlich robuster gegenüber Tokendiebstahl durch XSS-Angriffe sind als eine Speicherung im `localStorage`, auf den jedes im Browser laufende Skript zugreifen kann.

Positive Consequences

- Access- und Refresh-Token sind für clientseitiges JavaScript nicht auslesbar
- Cookies werden vom Browser automatisch bei jedem Request an dieselbe Origin mitgesendet
- Frontend muss die Tokens nicht selbst verwalten oder in den Application-State laden

Negative Consequences

- Erfordert zusätzlichen CSRF-Schutz, da Cookies vom Browser auch bei ungewollten Cross-Site-Requests automatisch mitgesendet werden (siehe ADR-008)
- `SameSite=Lax` und Cookie-Pfad müssen sorgfältig konfiguriert werden

Pros and Cons of the Options

httpOnly-Cookie

- Gut, weil JavaScript im Browser keinen Lesezugriff auf den Token hat
- Gut, weil kein manuelles Token-Handling im Frontend notwendig ist
- Nicht gut, weil zusätzlicher CSRF-Schutz erforderlich wird

localStorage

- Gut, weil der Zugriff aus dem Frontend-Code einfach ist
- Nicht Gut, weil jedes im Browser ausgeführte Skript (auch über XSS eingeschleuster Code) den Token auslesen könnte
- Nicht Gut, weil kein automatisches Mitsenden bei Requests erfolgt, sondern der Token manuell in jeden Request-Header eingefügt werden müsste

Market Adoption httpOnly-Cookies gelten als etablierte Best Practice zur Speicherung sicherheitskritischer Tokens im Browser und werden von gängigen Sicherheitsrichtlinien (z. B. OWASP) empfohlen.

Consequences Der `AuthCookieService` setzt Access-Token und Refresh-Token als httpOnly-Cookies; ein zusätzlicher, nicht-httpOnly `csrfToken`-Cookie wird für das Double-Submit-Pattern benötigt (siehe ADR-008).

Links

- Related to ADR-008
-

A.1.8. ADR-008 CSRF-Schutz über Double-Submit-Cookie-Pattern

Status: accepted

Deciders: Team

Date: 2026-05-27

Context and Problem Statement Da Authentifizierungs-Cookies vom Browser automatisch bei jedem Request an dieselbe Origin mitgesendet werden (siehe ADR-007), besteht ein Risiko für Cross-Site-Request-Forgery. Wie soll das Backend zustandsändernde Requests gegen CSRF absichern?

Decision Drivers

- Zustandslosigkeit der REST-API (kein serverseitiger Session-Speicher)
- Kompatibilität mit dem httpOnly-Cookie-Ansatz aus ADR-007
- Unabhängigkeit vom verwendeten Frontend-Framework

Considered Options

- Double-Submit-Cookie-Pattern mit X-CSRF-Token-Header
- Klassisches serverseitiges CSRF-Token mit Formularbindung (Standard-Spring-Security-CSRF)

Decision Outcome **Chosen option:** Double-Submit-Cookie-Pattern, weil es zum zustandslosen REST-API-Ansatz passt, keinen serverseitigen Session-Speicher benötigt und unabhängig vom Frontend-Framework funktioniert, solange dieses den Cookie-Wert auslesen und als Header mitsenden kann.

Positive Consequences

- Kein serverseitiger Session-Speicher für CSRF-Tokens notwendig
- Funktioniert unabhängig vom Frontend-Framework
- Sichere Methoden (GET, HEAD, OPTIONS) sowie Login/Register/Refresh sind explizit von der Prüfung ausgenommen, da hierfür noch kein gültiger CSRF-Cookie existieren kann

Negative Consequences

- Frontend muss den csrfToken-Cookie aktiv auslesen und manuell als X-CSRF-Token-Header mitschicken
- Zusätzliche Filterkomponente (CsrfValidationFilter) im Request-Pfad notwendig

Pros and Cons of the Options

Double-Submit-Cookie-Pattern

- Gut, weil es ohne Server-Session auskommt
- Gut, weil es sich gut mit einer SPA kombinieren lässt
- Nicht gut, weil das Frontend aktiv am Header-Handling beteiligt sein muss

Klassisches Spring-Security-CSRF mit Formularbindung

- Gut, weil es eine Standardlösung von Spring Security ist
- Nicht gut, weil es eher auf serverseitig gerenderte Formulare statt auf eine SPA mit JSON-API ausgelegt ist
- Nicht gut, weil die Integration mit einem reinen REST/JSON-Ansatz zusätzlichen Anpassungsaufwand erfordert hätte

Market Adoption Das Double-Submit-Cookie-Pattern ist ein anerkanntes, von OWASP dokumentiertes Verfahren zum CSRF-Schutz in zustandslosen APIs.

Consequences Der `CsrfValidationFilter` vergleicht bei jedem nicht-sicheren Request den `csrfToken`-Cookie-Wert mit dem `X-CSRF-Token-Header`; bei fehlendem oder abweichendem Wert wird der Request mit HTTP 403 abgelehnt.

Links

- Related to ADR-007
-

A.1.9. ADR-009 Persistente lokale Medienablage für Produktbilder

Status: accepted

Deciders: Team

Date: 2026-05-27

Context and Problem Statement Produktbilder müssen von Admins hochgeladen und dauerhaft, auch über Container-Neustarts hinweg, gespeichert werden. Wo sollen die Bilddateien abgelegt werden?

Decision Drivers

- Kein zusätzlicher externer Anbieter mit eigenen Zugangsdaten/Kosten für ein Laborprojekt
- Einfachheit der Umsetzung innerhalb der bestehenden Docker-Compose-Infrastruktur
- Persistenz über Container-Neustarts hinweg

Considered Options

- Lokales Docker-Volume für Medien
- Cloud-Speicher/CDN (z. B. Amazon S3)

Decision Outcome **Chosen option:** Lokales Docker-Volume, weil für ein Laborprojekt kein externer Anbieter mit zusätzlichen Kosten und Zugangsdaten benötigt wird und ein Docker-Volume mit vertretbarem Aufwand dauerhafte Persistenz sicherstellt.

Positive Consequences

- Keine externen Zugangsdaten oder Kosten notwendig
- Einfache Integration über ein benanntes Docker-Volume (`mavazihub_media`)
- Bilder bleiben über `docker compose down/up` hinweg erhalten

Negative Consequences

- Bilder sind an das Volume auf dem jeweiligen Host gebunden, keine automatische Verteilung über mehrere Standorte
- Keine CDN-Vorteile wie geografisch verteilte Auslieferung

Pros and Cons of the Options

Lokales Docker-Volume

- Gut, weil keine externen Abhängigkeiten entstehen
- Gut, weil die Umsetzung mit vorhandenen Bordmitteln (Docker-Volumes) einfach ist
- Nicht gut, weil keine horizontale Skalierung/Verteilung möglich ist

Cloud-Speicher/CDN

- Gut, weil Bilder geografisch verteilt und performant ausgeliefert würden
- Nicht gut, weil ein externer Vertrag/Account und Zugangsdaten benötigt würden
- Nicht gut, weil zusätzliche Kosten und Komplexität entstehen, die für ein Laborprojekt nicht gerechtfertigt sind

Market Adoption Lokale Volume-basierte Speicherung ist für kleinere Anwendungen und Laborprojekte üblich, während produktive E-Commerce-Systeme in der Regel auf Cloud-Speicher/CDN setzen.

Consequences Ein Wechsel auf einen Cloud-Anbieter würde primär `ProductImageStorageService` betreffen, ohne andere Module anzufassen (siehe 3.5 Erweiterbarkeit).

Links

- Related to ADR-005

A.1.10. ADR-010 Swagger/OpenAPI für interne API-Dokumentation

Status: accepted

Deciders: Team

Date: 2026-05-27

Context and Problem Statement Die REST-API muss während der Entwicklung und für die Abnahme nachvollziehbar dokumentiert und testbar sein. Wie soll die API-Dokumentation bereitgestellt werden?

Decision Drivers

- Aktualität der Dokumentation gegenüber dem tatsächlichen Code
- Möglichkeit, Endpunkte direkt während der Entwicklung zu testen
- Geringer Pflegeaufwand für das Team

Considered Options

- Swagger/OpenAPI (automatisch generiert)
- Manuell gepflegte API-Dokumentation (z. B. separates Dokument)

Decision Outcome **Chosen option:** Swagger/OpenAPI, weil die Dokumentation automatisch aus dem Code generiert wird und dadurch immer mit dem tatsächlichen Stand der API übereinstimmt, was bei einer manuell gepflegten Dokumentation nicht garantiert werden könnte.

Positive Consequences

- Immer aktuelle Schnittstellenübersicht ohne manuellen Pflegeaufwand
- Direktes Testen von Endpunkten über die Swagger-UI während der Entwicklung möglich
- Erleichtert die Abstimmung zwischen Backend- und Frontend-Team

Negative Consequences

- Swagger-Endpunkte (`/swagger-ui/**`, `/v3/api-docs/**`) müssen bewusst öffentlich zugänglich bleiben, was im Produktivbetrieb zusätzlich abzusichern wäre
- Dient ausschließlich als internes Entwicklungs-/Dokumentationswerkzeug, nicht als öffentliche Kundendokumentation

Pros and Cons of the Options

Swagger/OpenAPI

- Gut, weil die Dokumentation automatisch aktuell bleibt
- Gut, weil Endpunkte direkt in der UI getestet werden können
- Nicht gut, weil die Endpunkte zusätzlich abgesichert werden müssten, falls das System produktiv betrieben würde

Manuell gepflegte Dokumentation

- Gut, weil die Darstellung frei gestaltbar wäre
- Nicht gut, weil sie schnell veraltet, wenn Code-Änderungen nicht konsequent nachgepflegt werden
- Nicht gut, weil zusätzlicher manueller Aufwand ohne direkten Mehrwert für die Entwicklung entsteht

Market Adoption OpenAPI/Swagger ist der De-facto-Standard zur Dokumentation von REST-APIs im Java/Spring-Umfeld und wird breit von Entwicklungswerkzeugen unterstützt.

Consequences Die Swagger-Pfade sind in `SecurityConfig` explizit als `permitAll()` freigegeben; bei einem produktiven Einsatz müsste dies überprüft und ggf. eingeschränkt werden.

Links

- Related to ADR-002
-

A.1.11. ADR-011 Keine echte Zahlungs- und Versandanbindung (Simulation)

Status: accepted

Deciders: Team

Date: 2026-05-27

Context and Problem Statement Der Checkout-Prozess erfordert im Pflichtenheft eine Zahlungsabwicklung. Soll ein echter Zahlungsanbieter bzw. Versanddienstleister integriert werden, oder wird der Vorgang simuliert?

Decision Drivers

- Fokus des Projekts liegt auf den fachlichen Kernprozessen (Katalog, Warenkorb, Bestellung, Rücksendung, Rollen)
- Echte Zahlungs-/Versandanbindung erfordert reale Verträge, Zugangsdaten und rechtliche Prüfung
- Begrenzter Projektzeitraum eines Laborprojekts

Considered Options

- Simulierte Zahlung (`SIMULATED_PAID`) ohne echten Anbieter
- Integration eines echten Zahlungsanbieters (z. B. Stripe/PayPal) und Versanddienstleisters

Decision Outcome **Chosen option:** Simulierte Zahlung ohne echten Anbieter, weil eine echte Integration reale Verträge, Zugangsdaten und rechtliche Prüfungen erfordert hätte, die den Rahmen und die Zeitplanung eines studentischen Laborprojekts sprengen, während der fachliche Kernprozess (Bestellauslösung, Bestandsreduktion, Statuswechsel) davon unabhängig vollständig demonstrierbar bleibt.

Positive Consequences

- Kein Abhängigkeit von externen Verträgen, Testkonten oder Zugangsdaten
- Kernprozess der Bestellabwicklung bleibt vollständig testbar und demonstrierbar
- Kein Umgang mit echten Zahlungsdaten, dadurch geringeres Sicherheits-/Compliance-Risiko im Projektkontext

Negative Consequences

- Der Bestellstatus wird direkt nach Checkout ohne echten Zahlungsvorgang auf PAID gesetzt
- Das System ist in diesem Punkt nicht produktionsreif und müsste vor einem echten Einsatz um eine reale Zahlungs-/Versandanbindung erweitert werden

Pros and Cons of the Options

Simulierte Zahlung

- Gut, weil kein externer Anbieter benötigt wird
- Gut, weil der Fokus auf den fachlichen Kernprozessen bleibt
- Nicht gut, weil das System in diesem Punkt nicht produktionsreif ist

Echte Zahlungsanbindung

- Gut, weil ein realistischer End-to-End-Prozess entstünde
- Nicht gut, weil reale Verträge, Zugangsdaten und rechtliche Prüfung notwendig wären
- Nicht gut, weil der Aufwand in keinem Verhältnis zum Projektzeitraum steht

Market Adoption In produktiven E-Commerce-Systemen ist die Integration etablierter Zahlungsanbieter (z.B. Stripe, PayPal, Adyen) Standard; für Labor- und Lernprojekte ist eine simulierte Zahlung ein gängiges, akzeptiertes Vorgehen.

Consequences Diese Einschränkung wird in Kapitel 6.6 (Bekannte Einschränkungen, Person 4) sowie im Ausblick (Kapitel 7.3) als konkreter Erweiterungspunkt genannt.

Links

- Related to ADR-002